

A Configurable RISC-V Co-Processor with Instruction-Controlled Stream-Based Accelerators

ROHAN KRISHNA VIJAYARAGHAVAN, Technische Universität Dresden, Germany

AHMED KAMALELDIN, Technische Universität Dresden, Germany

MATTHIAS NICKEL, Technische Universität Dresden, Germany

LESTER KALMS, Center of Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Technische Universität Dresden, Germany

DIANA GÖHRINGER, Center of Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Technische Universität Dresden, Germany

The increasing use of Deep Neural Network (DNN)-based applications on edge devices has imposed new computing challenges, as such workloads demand high computational capabilities while ensuring low power consumption. To address these challenges, enhancing the General-Purpose (GP) computing units of edge devices by integrating hardware accelerators with application-specific functions provides the flexibility to support diverse workloads, thereby improving computing efficiency. This paper proposes a modular RISC-V coprocessor capable of hosting multiple stream-based accelerator IPs in a scalable platform. A low-latency interconnect interfaces the IPs with a local multi-bank scratchpad memory and supports direct IP-to-IP communication. A custom instruction set complemented by software macros is used by a RISC-V core for configuration, execution, and memory management in the coprocessor through the open-source eXtension Interface. Decoupling the coprocessor platform from the core clock domain using asynchronous FIFOs shows that the coprocessor supports integration of up to 8 custom IPs and 16 memory banks at a frequency of 294-333 MHz, when prototyped on an AMD/Xilinx RFSoc 4x2 evaluation board, while consuming 3% or less of the LUTs available on the board. A ShuffleNet-V2 case study shows that the coprocessor achieves comparable latency and up to 51% lower resource utilization compared to standalone HLS IPs.

Additional Key Words and Phrases: RISC-V, Hardware Accelerators, High-Level Synthesis, FPGAs

1 Introduction

The rapid proliferation of Machine Learning (ML)-driven applications on edge devices has intensified the demand for novel computing methodologies that deliver high computing performance while maintaining low power consumption. Currently, dedicated ML-based accelerators for Deep Neural Networks (DNNs) and computer vision are embedded on edge devices. These dedicated accelerators enable offloading compute-intensive workloads from General Purpose (GP) cores, thereby enhancing both computational performance and energy efficiency [12].

Nowadays, heterogeneous System on a Chip (SoC) architectures increasingly adopt RISC-V-based cores alongside diverse hardware accelerators, driven by the momentum of open-source hardware and software projects

Authors' Contact Information: Rohan Krishna Vijayaraghavan, Technische Universität Dresden, Dresden, Germany, rohan_krishna.vijayaraghavan@tu-dresden.de; Ahmed Kamaleldin, Technische Universität Dresden, Dresden, Germany, ahmed.kamal@tu-dresden.de; Matthias Nickel, Technische Universität Dresden, Dresden, Germany, matthias.nickel@tu-dresden.de; Lester Kalms, Center of Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Technische Universität Dresden, Dresden, Germany, lester.kalms@tu-dresden.de; Diana Göhringer, Center of Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Technische Universität Dresden, Dresden, Germany, diana.goehringer@tu-dresden.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1936-7414/2026/5-ART

<https://doi.org/10.1145/3816253>

that enable rapid development and prototyping in open-hardware research. However, the design and development of heterogeneous SoC architectures remain time- and cost-intensive, particularly when targeting application domains that demand multiple heterogeneous accelerators to handle diverse workloads [13]. Moreover, many existing heterogeneous SoC architectures lack the modularity and configurability to adapt internal components such as cores, memory, and interconnects to support external accelerators.

As a result, several open-hardware research platforms have been introduced as reusable and configurable solutions for integrating and supporting heterogeneous domain-specific accelerators alongside GP cores [17, 19]. Those platforms promote the reusability and flexibility of the hardware architecture, which can be adjusted to meet a diverse range of workload requirements across application domains. Moreover, open-hardware platforms offer the flexibility to support various accelerator coupling techniques [7] with RISC-V cores, either as loosely coupled memory-mapped peripherals or as tightly coupled pipeline extensions. However, programmability and system management in such platforms are often left to the developer to create custom mechanisms for communication and data sharing between custom accelerators and GP cores. This creates a gap between the flexibility and modularity offered by the hardware and the lack of software and management support [1]. For instance, low-level drivers can control loosely coupled accelerators as memory-mapped peripherals. Still, the developer must manage critical aspects, such as data sharing and coherence, according to the target application domains.

Tightly coupled accelerators address some of these issues, replacing drivers and API calls with custom instructions, allowing accelerator functionality to be invoked as a software-level kernel while incurring minimal software overhead [7]. Although open-source configurable interfaces such as the eXtension Interface (XIF) [23] and the Rocket Custom Co-processor (RoCC) interface [2] provide interfaces to tightly couple accelerators to GP cores, few works in the state-of-the-art provide direct plug-and-play platforms for accelerators as co-processors. Further, to the best of our knowledge, no other work provides optimized support for integrating stream-based accelerators with instruction-level software control over the accelerator kernels.

Therefore, a modular and configurable RISC-V-based co-processor capable of hosting a scalable number of stream-based accelerators with configurable memory size was presented in our previous work [30]. The objective is to transform stream-based macro-kernels to the granularity of micro-operations that can be handled and executed through custom instructions. Our co-processor demonstrates an abstraction-based approach to extending a RISC-V core with custom accelerators without requiring additional pipeline modifications, thereby showing the feasibility of seamlessly integrating diverse stream-based accelerators across different application domains and with varying characteristics. Also, a dedicated set of custom instructions is provided to manage the full accelerator operation flow (i.e., data loading, computation, data storing), thereby establishing a plug-and-play model for integrating heterogeneous accelerators. To further enhance scalability, the co-processor incorporates a modular low-latency interconnect that links accelerators with multiple scratchpad memory banks, enabling parallel execution and chaining of stream-based accelerators. This interconnect not only reduces communication overhead between memory and accelerators but also allows accelerators within the co-processor to operate concurrently and exchange intermediate results directly.

Our contributions in this work extend and improve upon our prior co-processor design presented in [30] as follows:

- In our previous work [30], we introduced a co-processor capable of integrating AXI-Stream-based IPs with local scratchpad memories, tightly coupled to a RISC-V core through the XIF interface [23]. In this work, we remove the reliance on XIF for runtime data transfer, instead mapping the co-processor scratchpad memory directly into the RISC-V core address space. Furthermore, the co-processor is decoupled from the core clock domain, enabling accelerators to operate at higher frequencies than the core.

- The prior design [30] featured a modular interconnect with dedicated data movement controllers, and this work refines and optimizes these internal components, resulting in lower resource utilization and improved programmability.
- Building on our previously proposed custom RISC-V instructions and unified software macros for accelerator control [30], this work validates the proposed architecture using a ShuffleNet-V2 [22] as a DNN workload, demonstrating the ability of the co-processor to accelerate full end-to-end workloads.

The rest of the paper is structured as follows: Section 2 discusses background and related work. Our co-processor microarchitecture implementation is introduced in Section 3. Programming methodology and software support are discussed in Section 4. An evaluation and experimental results are presented and analyzed in Section 5. Finally, Section 6 concludes this work.

2 Related Work

This section surveys state-of-the-art research on frameworks and architectures for the rapid prototyping and integration of accelerators into heterogeneous RISC-V-based SoCs, covering tool flows, platform sockets, and co-processors. Table 1 compares the existing literature in two categories, customizable SoC development platforms and co-processor designs. The related work is expanded upon in the following subsections, followed by a concluding discussion.

SoC Development Platforms

TaPaSCo [10] and ParallelXL [6] provide toolflows for integrating custom IP cores developed through High-Level Synthesis (HLS) or Hardware Description Language (HDL) design methodologies into heterogeneous systems. Both frameworks also offer run-time Application Programming Interfaces (APIs) for interfacing and communicating with hardware accelerators through GP cores (e.g., ARM or RISC-V). Further, ParallelXL includes a work-stealing mechanism for load balancing and support for nested and dynamic task parallelism.

Similar to TaPaSCo [10] and ParallelXL [6], the ESP platform [19] provides an open-source environment for heterogeneous SoC design, but it features a modular tile-based architecture for processors and accelerators. It follows a socket-based approach that encapsulates the contents of each tile, thereby decoupling it from the rest of the system. This platform has been extended in [9] to enable seamless integration of third-party IPs by augmenting the socket with additional HW/SW interfaces.

The HERO platform [15] implements a RISC-V-based many-core system on the Field Programmable Gate Array (FPGA) fabric. In contrast to TaPaSCo [10], ParallelXL [6] and the ESP platform [19], the HERO platform has a stronger focus on the Linux software stack running on the SoC's ARM host CPU. Building on this platform, [3] modifies the SoC cluster to support plug-and-play integration of custom accelerators, introducing an overlay with a customizable wrapper for IPs defined in HLS or HDL.

Maisto et al. propose Simply-V [18], a research platform for heterogeneous RISC-V-based SoCs, targeting open-hardware research and rapid prototyping on FPGAs. The platform follows a plug-and-play approach, enabling straightforward integration of hardware accelerators through a dedicated framework. To support this flexibility, the architecture employs a hierarchical multi-bus system. At its center is the Main Bus (MBus), which interconnects the main components—including the RISC-V core, debugging units, monitors, on-chip memories, co-processors/accelerators, and subordinate buses. The subordinate buses consist of the High-bandwidth Bus (HBus), which links the MBus to DDR or High Bandwidth Memory (HBM) and also supports direct attachment of accelerators, and the Peripheral Bus (PBUS), which provides access to peripherals such as UART. To demonstrate the efficiency of their approach, the authors evaluated multiple system configurations with various benchmark kernels, memory systems, and soft-cores.

Table 1. Overview of Related Work.

Reference	Support of Custom Accelerators	Accelerator: Programmability	Accelerator: Interconnect / Architecture	Model/ Granularity	Accelerator: Memory Integration	Target Flow
SoC Development Platform						
TaPaSCo [10]	✓	API	Clusters over bus connected	Task-Parallel	Local Memory	FPGA
ParallelXL [6]	✓	API	Clusters over bus	connected Task-Parallel	L1 or Stream-Buffers, Shared LLC	FPGA
ESP [9, 19]	✓	API	NoC	System Level	Private Cache and Local Memory ¹	FPGA
HERO [3, 15]	✓	API	Cluster connected over bus	Task-Parallel	Scratchpad	FPGA
X-HEEP [17, 29]	✓	API	Bus	System Level	-	ASIC, FPGA
Simply-V [18]	✓	API	Bus	System Level	-	FPGA
Co-Processor						
FDRA [27]	✓	Instruction, API	CGRA	Instruction Level, Task-Parallel	Multi-bank Scratchpad	ASIC, FPGA
Spatz [25]	✗	Instruction	VPU	Instruction Level	Multi-bank VRF	ASIC
TYRCA [8]	✗	Instruction	Fixed Accelerators	Instruction Level	-	FPGA
NM-Carus ² [5]	✗	Instruction	VPU	Instruction Level	NMC	ASIC
Quadrilatero [4]	✗	Instruction	Fixed SA	Instruction Level	Multi-bank MRF	ASIC
Co-Processor Development Platform						
Our Work	✓	Instruction	Tree-based	Instruction Level, Kernel Chaining	Multi-bank Scratchpad	FPGA

X-HEEP [17, 29] is a single-core RISC-V-based microcontroller development and prototyping platform. All components are connected via the Open Bus Interface (OBI) interconnect, enabling the integration of custom peripherals and accelerators. In addition, X-HEEP supports a directly connected co-processor to the RISC-V CPU through the XIF [17]. Its primary design flow targets Application Specific Integrated Circuit (ASIC) implementations, but also supports FPGAs.

The presented SoC development platforms [6, 10, 15, 17–19] primarily focus on the plug-and-play integration of loosely coupled accelerators. Although X-HEEP [17] supports tightly coupled co-processor integration via the XIF, the platforms themselves does not provide a plug-and-play approach for the direct use of custom co-processors.

Co-Processor

Spatz [25] is a VPU based on the RISC-V Vector Extension (RVV), targeting low-power edge applications. It serves as a co-processor to Snitch [31], connected via the XIF. The Spatz architecture consists of a multi-bank VRF, vector units for load/store operations, arithmetic, and sliding operations, and a controller. Evaluation results report a speedup of up to 6.8× (conv2d) compared to the Snitch-only baseline. Spatz-MX [26] extends Spatz by reusing the existing VRF, introducing new configuration instructions, and adding hardware features such as a broadcast block near the Floating Point Units (FPUs) to increase data reuse and a latch-based buffer for tile results in the Vector Function Unit (VFU), instead of adding a dedicated MRF.

TYRCA [8] is a tightly coupled RISC-V-based co-processor over the XIF. The co-processor accelerates Post-Quantum Cryptography (PQC) based on the Hamming Quasi-Cyclic (HQC) scheme. The design integrates three specialized accelerators: the $\mathcal{R}$ -Unit (implementing the Karatsuba algorithm), a Reed-Solomon decoder (composed of Galois Field Reduce and Galois Field Carryless units), and Keccak. The reported speedup per accelerator ranges from 5.36× (Galois Field Carryless) up to 116.28× (Karatsuba), yielding an overall HQC acceleration of about 20× for HQC-128, HQC-192, and HQC-256. The co-processor introduces an area overhead, with increases of 26.3% in LUTs and 17% in registers, corresponding to less than 30% of the total SoC resources.

¹ESP Accelerator

²[5] proposes two variants of NMC elements: NM-Caesar and NM-Carus. We consider NM-Carus only here.

Caon et al. [5] propose a RISC-V-based NMC architecture for machine learning on edge devices. Two variants of the NMC unit are presented. The first, NM-Caesar, consists of a controller and an Arithmetic Logic Unit (ALU) connected to 2×16 KiB SRAM, targeting area efficiency. The second, NM-Carus, integrates a CV32E40X embedded CPU with 4×8 KiB SRAM, connected to a VPU via the XIF; it also includes a 512-bit register file and a configuration register, targeting performance. Across benchmark kernels, NM-Carus achieves up to $99.6\times$ higher throughput and up to $59.3\times$ better energy efficiency than the standalone RV32IMC main CPU. Compared to NM-Caesar, NM-Carus delivers $3.8\times$ higher throughput and $2.6\times$ better energy efficiency per output, at the cost of higher area overhead: approximately $2\times$ the area of a 32 KiB reference SRAM and 63.7% more area than NM-Caesar. Machetti et al. [17] evaluated NM-Carus on transformer and Convolutional Neural Network (CNN) models using the X-HEEP platform [29], demonstrating a $3.4\times$ speedup versus standalone CPU execution. With an early-exit strategy, the speedup increases to $5.4\times$ for transformers and $7.3\times$ for CNNs, yielding an energy reduction of $2.2\times$ to $3.6\times$.

Quadrilatero [4] incorporates a 4×4 SA as a co-processor into the CV32e40X via the XIF interface. The co-processor's area breakdown shows that the SA dominates resource usage at 82.8%, followed by the register file at 11.4%, the Load/Store Unit (LSU) and controller at 5.7%, and the permutation unit at 0.1%. Compared to a Spatz co-processor configuration with sixteen 32-bit FPUs, the proposed SA achieves similar performance while reducing energy consumption by 6% and area by 34%. Relative to Spatz [25] and Spatz-MX [26] with four FPUs each, the SA attains approximately $3.9\times$ higher performance and 15% lower energy consumption, at the cost of 2.3-2.6 $\times$ higher area utilization.

FDRA [27] introduces a framework for mapping applications to a dynamically reconfigurable CGRA, implemented as a RISC-V co-processor based on the RoCC protocol [2].

Related work on co-processors [4, 5, 8, 25, 27] focuses on integrating domain- and application-specific hardware accelerators or improving the performance of specific operations, such as vector processing. Similar to our work, FDRA integrates a reconfigurable architecture as a co-processor. However, whereas FDRA implements a CGRA as a co-processor, which is a Domain-Specific Architecture (DSA), our goal is to provide a flexible, generic platform for investigating and prototyping RISC-V domain-independent co-processors that support custom streaming-based accelerators.

Discussion

Several works have advanced fast prototyping of heterogeneous SoCs [3, 6, 9, 10, 15, 19, 29]. Similar to Simply-V [18], we target plug-and-play-like integration of hardware accelerators for rapid FPGA-based prototyping. However, our approach also emphasizes ease of programming. It simplifies communication, configuration, and data movement between the CPU and accelerator. Thus, developers don't have to manage these details manually; they can simply use instructions instead of an API. While related co-processor proposals [4, 5, 8, 17, 25, 27] predominantly focus on operation-level functionality, our design supports not only standalone kernel execution but also the composition and chaining of multiple kernels within the co-processor execution flow. This combination of plug-and-play integration and easy programmability allows faster prototyping of hardware accelerators for FPGA based SoCs. Looking at the comparison in Table 1, our goal is to bring paradigms of the first group to the latter, providing a platform for rapid prototyping of customizable co-processors.

3 Implementation

This section presents the architecture of the proposed RISC-V-based co-processor. We outline how the co-processor is connected to the CV32E40X via the XIF, how custom instructions are offloaded and handled in the wrapper, and how a multi-banked scratchpad and LSU provide high-bandwidth data movement. We also describe the execution

unit with Compute Units (CUs) for stream-based IPs, support for arbitrary access patterns via an address queue, and the scalable single-cycle interconnect that links CUs to memory banks.

3.1 System Overview

In this work, an architecture capable of hosting multiple stream-based hardware accelerator IP (described in either HDL or HLS) is implemented as a co-processor to a RISC-V core using Verilog HDL. The resulting SoC is shown in Figure 1, featuring the co-processor tightly coupled with the XIF interface of the CV32E40X [24] RISC-V core through the XIF protocol. The core is interfaced with dedicated instruction and data memories through a 32-bit AXI4 interconnect. Further, instruction and data bridges are placed between the AXI4 interconnect and the CV32E40X core's native instruction/data (I/D) ports. These bridges, presented in [14], facilitate the integration of tightly coupled local memories into the core's address space, enabling low-latency memory access to the data stored in these memories.

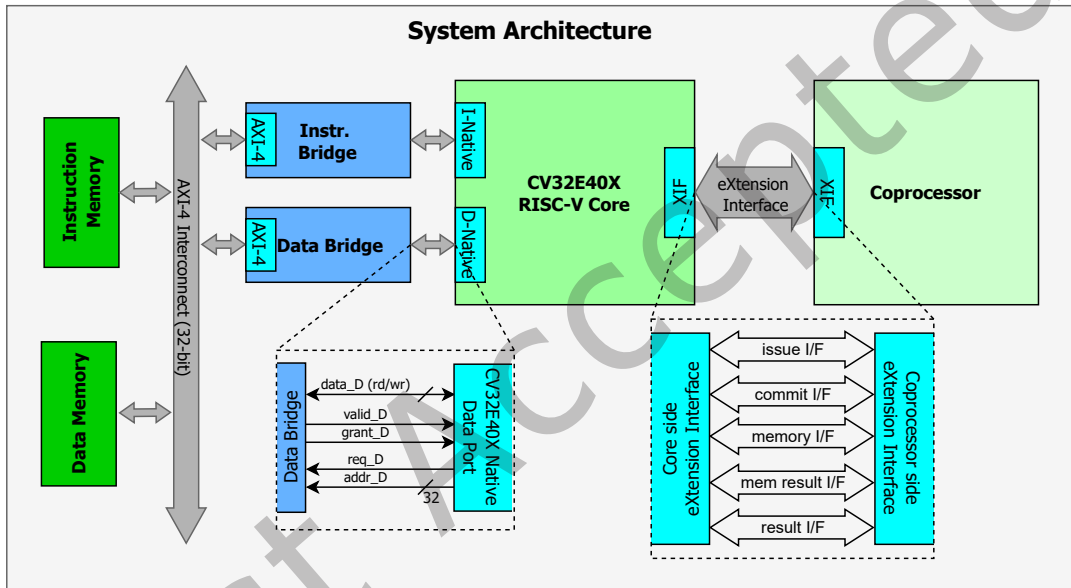


Fig. 1. Integration of the co-processor with a 32-bit RISC-V core in the proposed SoC, highlighting the core-memory and core-co-processor interface.

3.2 Tightly Coupled Co-processor Extension

This work leverages the XIF supported by the CV32E40X core to tightly couple the co-processor architecture with the 32-bit datapath of the RISC-V core. The co-processor is controlled and operated through custom RISC-V instruction extensions, which are offloaded to the co-processor via the XIF protocol. In particular, five interfaces of the XIF[23] are used by this work: *issue*, *commit*, *memory*, *memory result*, and *result*.

The *issue* interface allows the core to offload any instruction it classifies as illegal to the co-processor. When the custom instruction is an R-type RISC-V instruction, the core forwards the 32-bit instruction word along with the values of the two 32-bit registers referenced by the *rs1* and *rs2* fields. Once the co-processor accepts the offloaded instruction, the *commit* interface is used by the core to commit it, temporarily stalling the core pipeline

for the duration of the co-processor execution. The co-processor uses the *result* interface to signal completion of the offloaded instruction, allowing the core to resume normal execution. Additionally, the co-processor can return a 32-bit data word over the *result* interface, which the core writes to the register specified by the *rd* field of the custom instruction. Optionally, the co-processor may perform read or write operations through the *memory* and *memory result* interfaces, which provide direct access to the core's data memory. The overall logic handling the offloaded custom instructions is implemented inside the main wrapper of the co-processor as shown in Figure 2.

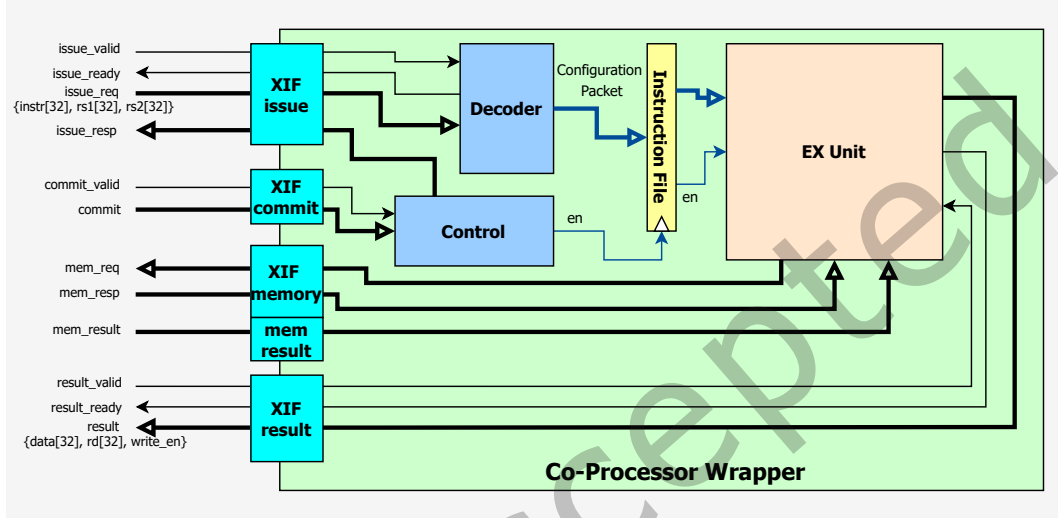


Fig. 2. Architecture of the co-processor wrapper, featuring the XIF, decoder and instruction controllers, and the EX unit which hosts the custom accelerators.

The operational components of the co-processor, such as the custom accelerator IPs and the local scratchpad memories, are abstracted into an *execute unit*, described in Section 3.4. Within the co-processor wrapper, a decoder interprets custom instructions offloaded by the core through the *issue* interface. Each decoded instruction is translated into a structured configuration format and stored, together with the two offloaded register values, in a local instruction file for later use by the execute unit.

An *instruction controller* manages all handshakes with the core via the XIF interfaces. Through the *issue* interface, it reports whether the co-processor supports the offloaded instruction and indicates whether it will perform memory transactions through the *memory* interfaces or produce a register write-back to the core. Upon receiving a commit through the *commit* interface, the controller triggers the execute unit to start processing stored instructions. Finally, it signals the core on completion and, if required, returns the write-back data through the *result* interface.

3.3 Co-processor Data Movement and Memory Access

This work employs a multi-banked scratchpad memory inside the co-processor to enable seamless, high-bandwidth memory access for the accelerator cores integrated in the co-processor. The number of memory banks and the depth of each bank are compile-time configurable, and each bank provides a 32-bit data interface. Every memory bank is implemented as a dual-port memory with dedicated read and write channels, exposing memory-mapped read and write interfaces to the accelerator IP.

In the initial version of this work, as presented in [30], the scratchpad memory was initialized with the data required for accelerator operation using custom instructions that leverage the XIF *memory* and *memory result* interfaces. In particular, four custom instructions - XLOADC, XLOAD, XSTOREC, and XSTORE - configure the co-processor with information such as source and destination addresses and array lengths. A dedicated LSU manages all data transfers between the core's data memory and the co-processor scratchpad via the XIF. The LSU interfaces the local scratchpad banks with the XIF memory interfaces, as shown in Figure 3.

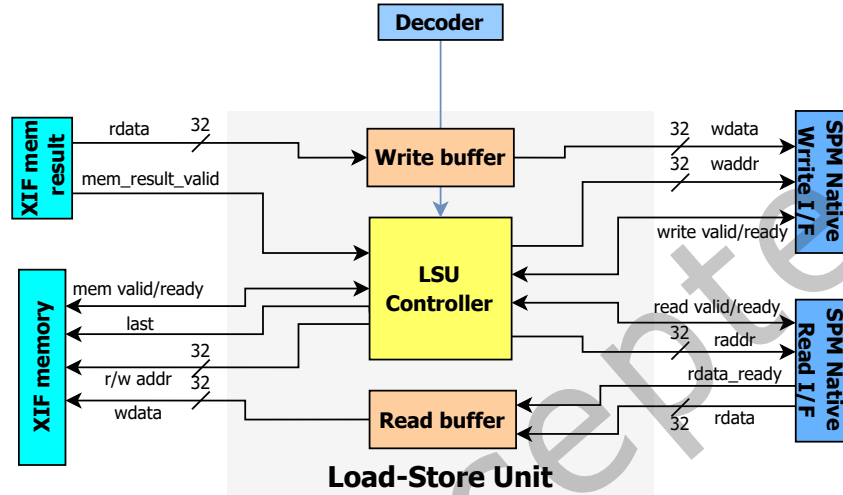


Fig. 3. LSU interface with the scratchpad memory bank and the XIF.

Each scratchpad memory bank exposes native read and write interfaces consisting of three main channels: a handshake channel, an address channel, and a data channel. The handshake signals are used for bank selection through address-based routing in a multi-bank setup, as discussed in Section 3.5. The LSU bridges these native signals to the distinct XIF interfaces: the *memory* interface is used for issuing both read and write transfer requests. In contrast, the *memory result* interface returns read data and write acknowledgments.

Because the XIF memory interface supports burst transfers of multiple 32-bit words in a single instruction, the LSU integrates two address generation units: one for the internal scratchpad bank addresses and another for the external data memory addresses. To maintain correct synchronization between the two sides in variable read latencies, the LSU also includes data buffers to hold incoming data from both the external and internal memories.

When the co-processor performs a load or store operation to transfer a data array between the core's data memory and the co-processor's scratchpad, it uses the custom instructions summarized in Table 2 to configure the LSU. In both cases, the LSU follows a statically scheduled sequence of operations implemented as a finite state machine (FSM) to synchronize data movement between the two memories. For a load operation, where the co-processor fetches an array of 32-bit words from the external data memory into the scratchpad, the XLOADC instruction first configures the LSU with the source address of the first word in external memory and the total transfer length. The XLOAD instruction sets the destination address inside the co-processor and starts the memory transaction. To begin, the LSU asserts a valid signal on the XIF *memory* interface and waits for a ready response from the core. Once acknowledged, the source address is sent through the *memory* interface, and the co-processor waits for the core to return the corresponding data word via the XIF *memory result* interface. The LSU stores this word in an internal buffer, then issues a write request to the selected scratchpad bank and, once accepted,

Table 2. LSU custom instructions.

Instruction	funct7	rs2	rs1	funct3	rd
XLOADC	-	Length	Source address	000	Status
XLOAD	-	-	Destination address	001	Status
XSTOREC	-	Length	Source address	010	Status
XSTORE	-	-	Destination address	011	Status

forwards the destination address and write data. The internal address counters are incremented, and the sequence repeats until the configured array length (given by XLOADC) is reached. For the final transfer, the LSU asserts a *last* signal on the *memory* interface to mark completion of the transaction.

Similarly, for store operations, the XSTOREC and XSTORE instructions configure the LSU and initiate the transfer of a data array from the co-processor's scratchpad to the core's external data memory. In this case, the LSU first issues a read request to the selected scratchpad bank, providing the source address. When the word is returned, it is placed into the internal data buffer and then forwarded to the core through the XIF *memory* interface, along with the destination address for the write. The core performs the write to external memory and returns an acknowledgment through the XIF *memory result* interface. The LSU increments its address counters and repeats this sequence until the entire array has been written. After completing either an XLOAD or XSTORE operation, the co-processor returns a status value of 1 to the core through the XIF *result* interface, indicating successful completion. If an error occurs on the core side during a memory access, the core signals an exception via the XIF *memory* interface; in this case, the operation is aborted, and a status value of 0 is returned.

The LSU in this work operates similarly to a direct memory access (DMA) controller, supporting direct transfer of data arrays between the core and the accelerator IP through the XIF using custom instructions. However, this approach presents certain limitations. First, the transfer of each 32-bit word through the XIF is not pipelined, introducing a fixed overhead for every word moved. For load operations, this overhead is approximately five clock cycles (four cycles on the core side and one cycle on the co-processor side). For store operations, there are also five cycles (two on the co-processor side and three on the core side). Additionally, the core pipeline stalls during these transfers because the core must explicitly perform the external memory accesses on behalf of the co-processor. As a result, the application incurs significant latency whenever it needs to provide inputs to or collect outputs from an accelerator IP. Furthermore, because the co-processor scratchpad memory is not mapped into the core's address space, the core cannot access this data during normal program execution. To address these drawbacks, the LSU in this work is extended to support a direct connection to the core's data bridge.

The data bridge proposed in [14] acts as a protocol converter, interfacing the core's native data port with the AXI-4 bus, and enables low-latency access from the core to tightly coupled local memories. It performs address-based multiplexing, forwarding requests from the core to the AXI-4 interconnect or local memories.

In this work, the LSU described above is extended to interface with the Tightly Coupled Data Memory (DTCM) port of the data bridge, connecting the co-processor's multi-banked scratchpad memory as a tightly coupled memory to the core, as shown in Figure 4. This extension's primary advantage is that it maps the scratchpad memory directly into the RISC-V core's address space. This integration provides several benefits. First, since the scratchpad is now directly accessible by the core without relying on custom instructions, data can be moved into the scratchpad during program initialization directly from the program memory, instead of the previous two-step process of first copying into the core's data memory and then issuing custom load/store instructions to move it again. This eliminates the redundancy of custom XLOAD/XSTORE instructions, which otherwise stalled the core pipeline. Second, because the scratchpad is now tightly coupled through the data bridge, the core achieves

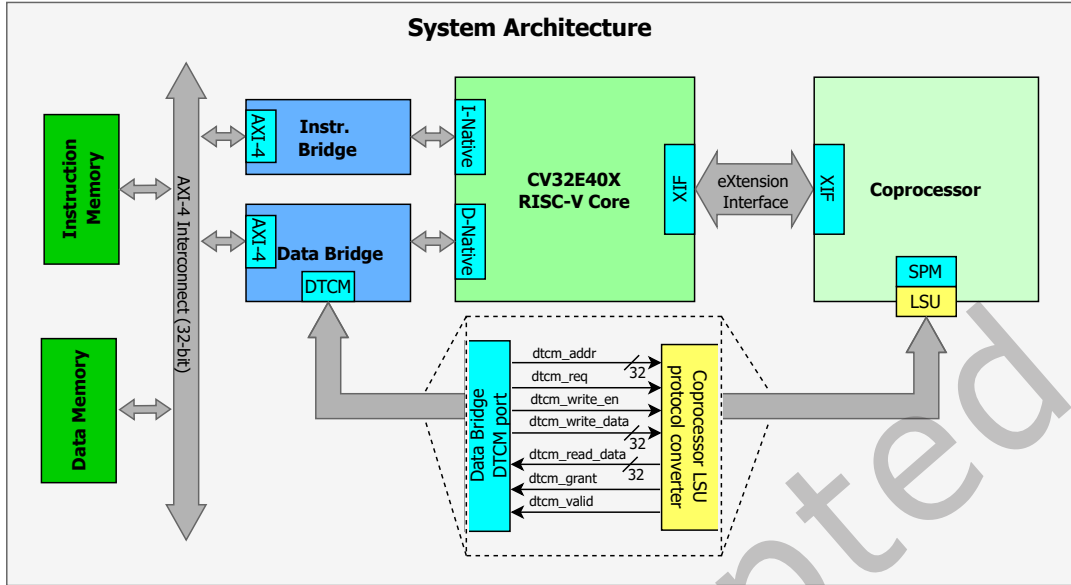


Fig. 4. Data bridge connecting the core's native data port to the AXI-4 interconnect and the co-processor scratchpad memory.

lower-latency access than the AXI-connected external memory. The extended LSU also bypasses its internal buffering in this mode and exposes the entire address space of the multi-banked scratchpad directly.

However, when the core needs to explicitly move large data arrays from external memory into the scratchpad during runtime, using the custom load/store instructions can still be advantageous. Although the scratchpad is memory-mapped, the latency per word transfer using XLOAD/XSTORE remains lower than issuing multiple standard RISC-V load/store operations.

3.4 Accelerators Wrapper – The Compute Unit

Custom accelerator IPs are integrated into the execution unit of the co-processor as a *compute cluster*, which hosts all the AXI-4 Stream-based IPs, coupled with the multi-banked scratchpad memory. This work dedicates a lightweight controller to each integrated IP to interface each stream-based IP with the memory-mapped ports of the scratchpad. This IP-controller pair is referred to as a CU in the remainder of this work.

The CU controller acts as a memory-mapped-to-stream controller, streaming data from a source address in the scratchpad into the input ports of the AXI-4 Stream IP, and collecting the output stream from the IP to write it back to a destination address in the scratchpad. The CU architecture, shown in Figure 5, illustrates the interfacing between the accelerator IP and the native memory bank ports.

When streaming data into the IP, the CU controller performs burst reads from the scratchpad and forwards data through an internal FIFO buffer. This FIFO decouples the memory-mapped interface from the IP's streaming input port, absorbing latency when the scratchpad read response is delayed or when the IP temporarily stalls its input. For collecting output data, the CU controller operates in the opposite direction: the IP acts as the master, driving the streaming handshake. In contrast, the CU forwards the data to the write ports of the scratchpad memory. To handle source and destination addressing, each CU controller integrates two address generation units, one for the read port (input streaming) and one for the write port (output streaming). Like the LSU, each CU is also controlled using a dedicated set of custom instructions, summarized in Table 3.

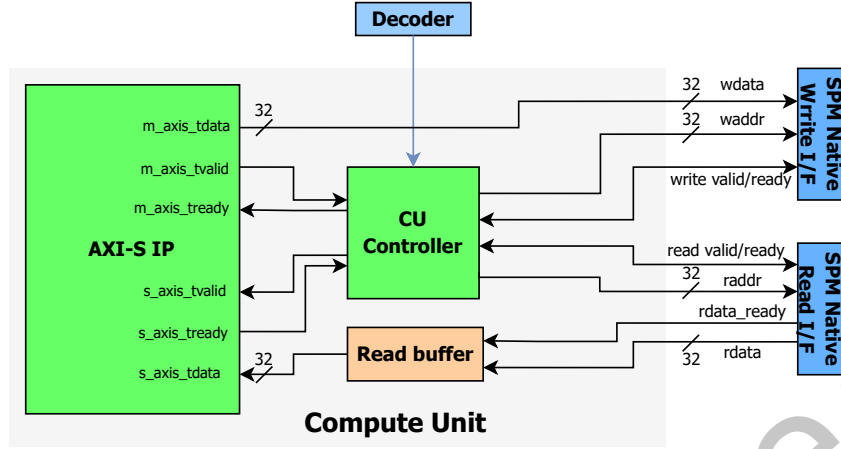


Fig. 5. Architecture of the CU, highlighting the CU-AXIS interface and the CU-SPM interface.

Table 3. Custom instructions to configure and control the CU.

Instruction	funct7	rs2	rs1	funct3	rd
XCFG	Target CU	Input length	Source address	100	Status
XCMP	Target CU	Output length	Destination address	101	Status

The XCFG instruction configures a CU with the source address of its input stream within the scratchpad memory and the number of 32-bit words to be streamed. The XCMP instruction sets the destination address for the output stream, specifies its length (to support IPs whose output size differs from the input), and triggers the CU to start the streaming operation. Since the co-processor can host multiple CUs, each CU is assigned a unique identifier. The target CU is selected by encoding its ID in the funct7 field of both XCFG and XCMP.

When the co-processor performs a compute operation by streaming an array of data through an IP, the XCFG instruction is first used to configure the CU controller with the source address and the input length. As with other configuration instructions, once these values are successfully written, the co-processor returns a status value through the XIF *result* interface. The XCMP instruction is then used to configure the output address counter of the CU and to begin the streaming operation. Unlike the LSU, the CU is designed to operate independently and does not stall the core pipeline during execution. Thus, a valid status is returned to the core immediately after configuration completes, after which the CU begins the streaming operation independently, freeing the core to continue executing subsequent instructions.

During execution, the CU initiates read requests to the scratchpad memory banks and fills its internal FIFO buffer with the returned data. Whenever data is present in the FIFO, the CU drives the handshake signals of the AXI4-Stream IP's input port and forwards the buffered words. This process continues until the input address counter reaches its programmed maximum. At the same time, the CU handles the IP's output stream, forwarding each valid output word to the write port of the scratchpad, and incrementing the output address counter accordingly. The transfer completes once the programmed output length is reached, which is explicitly specified by the XCMP instruction to support IPs that do not assert a *last* signal.

When the streaming operation completes, the CU returns to its idle state, but the core must explicitly determine that the transfer has finished. The XIF does not provide an interrupt or asynchronous notification mechanism for the co-processor to signal completion. Therefore, it is the responsibility of the core to probe the target CU and check whether it has finished execution. This is achieved using the XCFG instruction, where the core issues a new configuration request to the CU. If the CU is still busy with an ongoing streaming operation, it responds with a status value of "0". Once the CU has finished, the same instruction returns a status value of "1", indicating that the CU is idle and that the output data from the previous operation is now available.

An advantage of the input-output mapping methodology of the CUs is that an AXI4-Stream IP with multiple input or output ports can be interfaced with several CUs to operate effectively as a single accelerator. In this setup, each CU attached to the IP can be configured and triggered independently, and only the CUs connected to active output ports need to be probed for results. If a CU is connected to an unused input or output port, its streaming operation can simply be bypassed by setting the transaction length to "0" in its configuration instruction.

In the initial version of this work, as presented in [30], each CU controller was connected to a memory bank through a dedicated address port but could only be configured with a starting address and then read data sequentially. Consequently, any operation or IP that required non-contiguous data access, memory alignment, or rearrangement had to rely on the core to explicitly compute and issue individual memory addresses through standard instructions, which are costly and latency-heavy. To overcome this limitation, we extend the CU with an *address queue*, as shown in Figure 6. The address queue allows the core to preload a sequence of target addresses, one per data element, to be streamed into the AXI4-Stream IP. With this addition, the CU can fetch input data from arbitrary memory locations while still streaming it seamlessly into the IP, making the CU controller truly memory-mapped rather than bound to strictly sequential addressing.

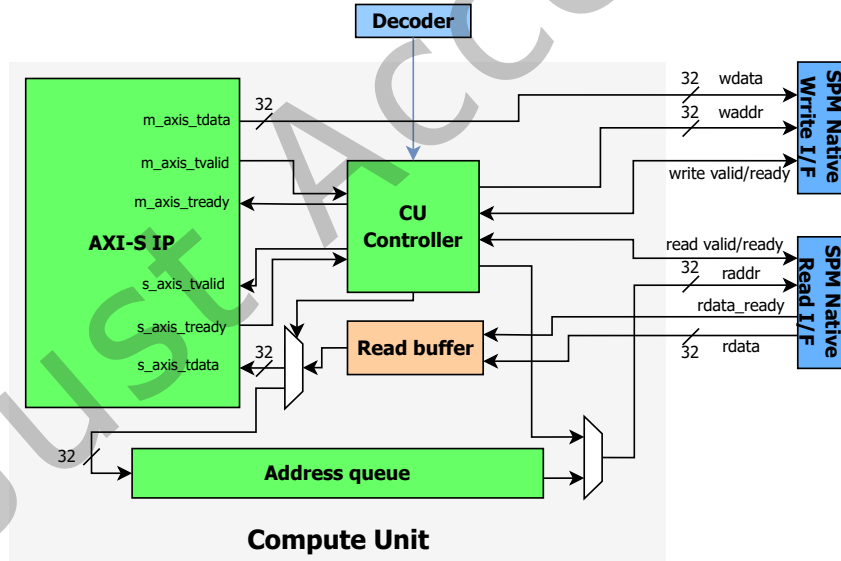


Fig. 6. Compute unit augmented with an address queue.

To accommodate the address queue in the CU, a multiplexer is placed on the *data* path coming from the Scratchpad Memory (SPM) read port, so that the streamed data can be directed either to the AXI4-Stream IP or into the CU's address queue. In the first phase (*address-load mode*), the core prepares a list of target addresses in

Table 4. ISA extension for arbitrary memory patterns in the augmented CU.

Instruction	funct7	rs2	rs1	funct3	rd
XLDA	Target CU	Input Length	-	110	Status
XCFGA	Target CU	Input Length	-	111	Status

the SPM; the CU then streams these words from the SPM into the address queue (via the mux) instead of into the IP. In the second phase (*execution mode*), the CU uses the values popped from the address queue as read addresses on the SPM interface; the returned data words are forwarded along the standard path to the AXI4-Stream IP. This way, arbitrary access patterns can be preloaded, and the corresponding data can then be streamed to the IP without computing addresses on the fly.

To enable this augmented functionality of the CU, two additional custom instructions are introduced into the ISA extension, as shown in Table 4. The XLDA instruction is used together with the existing XCFG instruction to load a sequence of addresses into the CU, as part of the *address-load mode*. First, XCFG configures the CU with the source location of the address sequence stored in the SPM. When the core then issues XLDA to the target CU, the input-data multiplexer is switched toward the address queue, and the CU streams the words from the SPM into this queue, similar to how XCMP streams data to an IP. Once the address sequence is fully loaded, and the application is ready to begin the actual streaming operation as part of the *execution mode*, the XCFGA instruction configures the CU to fetch read addresses from the address queue instead of using its internal sequential address counter. After this step, the standard XCMP instruction can be used to perform the streaming operation through the AXI4-Stream IP, with data now fetched from the arbitrary addresses stored in the queue.

3.5 Scalable Interconnect Network

The previous subsections described the individual controllers that move data to the SPM and support the AXI4-Stream IPs during streaming. This section shows how the co-processor hosts multiple AXI4-Stream IPs within the execution unit and interfaces them with a multi-bank SPM. All CUs and memory banks are connected through a single-cycle interconnect network, as presented in [30], and illustrated in Figure 7.

Figure 7 illustrates a 4×4 example in which the memory banks are connected in a *slave tree* and the CUs and the LSU are connected in a *master tree*. The interconnect scales to an $N \times M$ topology, where the master tree hosts N master units and the slave tree hosts M slave units. Between the two trees lies a logarithmic *router-arbiter* interconnect. Every master unit has a connecting path to every slave unit and can access them via each slave's unique address range. Each CU in the master tree is assigned a compile-time *CU ID* used for control through custom instructions, and the LSU is also part of the master tree and is assigned a default ID of 0.

The first stage of the interconnect network is the routing tree, which is connected to the master tree. The routing tree consists of multiple levels of routing switches, each with independent datapaths for read and write. The number of levels is $\log_2 M$, where M is the number of slave units. Each first-level switch connects to a corresponding master unit, and the last level connects to the arbitration tree. A routing switch forwards an input read or write request to one of two output directions, as shown in Figure 8, determined by the target address of the slave unit in the incoming request. The incoming address is partitioned into (i) the memory bank's *internal address* and (ii) the following $\log_2 M$ *bank-select* bits. For example, with $M=8$ banks (depth 1024 words), the lowest 10 bits form the internal address, and the next 3 bits select the bank. At each level, the bank-select bits are compared against that level's *tree mask*; the comparison result determines the output direction. This way, requests traverse successive levels of the routing tree to form a path from the master tree to the arbitration tree.

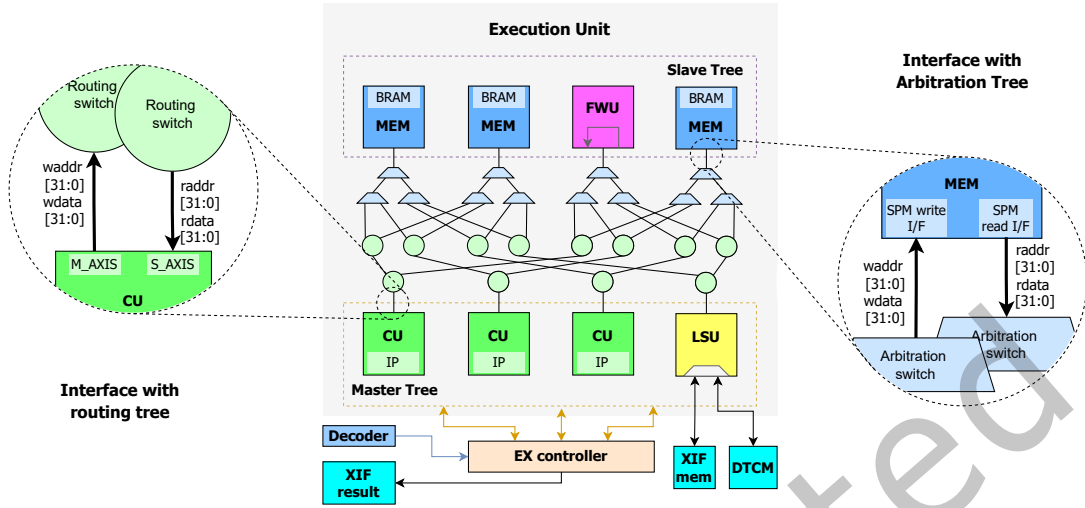


Fig. 7. The Execution unit featuring the low-latency interconnect (4x4 as an example here). The execution unit communicates with the core through the EX controller, and with the external memory through the LSU. The interfaces between the interconnect and the master & slave trees are highlighted.

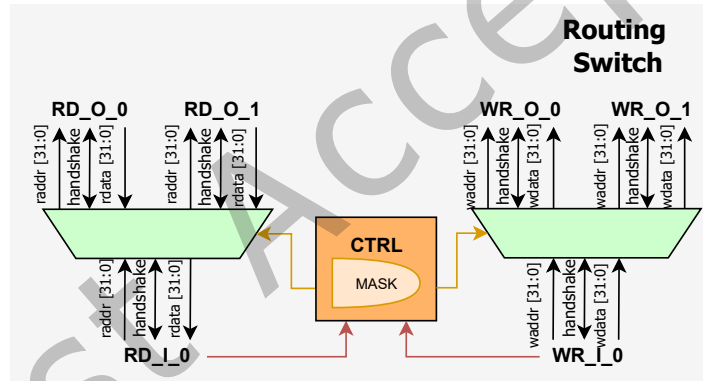


Fig. 8. Architecture of the routing switch.

The arbitration tree forms the second stage of the interconnect, adjacent to the slave tree and composed of multiple levels of two-input arbitration switches. As with the routing switches, each arbitration switch implements independent datapaths for the read and write interfaces. The tree depth is $\log_2 N$, where N is the number of master units. Switches at the last level connect one-to-one to the slave units, while the first level connects to the last level of the routing tree, ensuring a possible path from every master to every slave.

Unlike the routing switches, the arbitration switch forwards one of two incoming requests to the output, as shown in Figure 9, and therefore includes a small controller. If only a single request is present, it is forwarded immediately to the output. However, in the case of two simultaneous requests, a round-robin flag toggles each cycle, granting alternating service to the two inputs to avoid starvation. Because the read and write datapaths are independent, arbitration on the read path does not affect the write path, and vice versa.

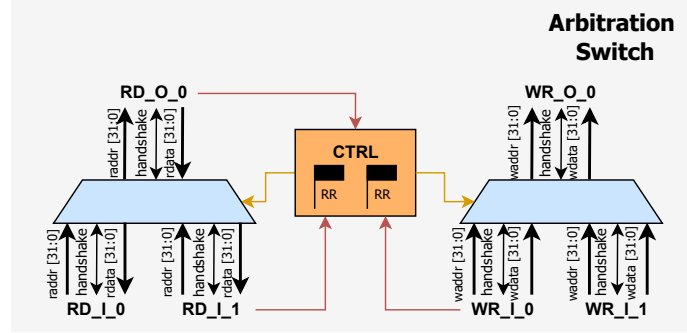


Fig. 9. Architecture of the arbitration switch.

Thus, requests propagate from the first level of the arbitration tree to the last, establishing a complete path from the master units to the slave units. With this mechanism, multiple paths can be active concurrently across the read and write layers; however, in any given cycle, a master may access at most one slave, and each slave may be accessed by at most one master. When a path is active, the master can issue a read, where the address is forwarded to the slave and the returned word flows back along the same path, or a write, where the master forwards both address and data, and the slave returns an acknowledgment.

In this scheme, each master unit may initiate a request to a slave unit, but the interconnect does not support direct master-to-master communication. Since the targets are stream-based IPs, the output stream of one IP is often intended as the input to another. If all slave units are memory banks, such IP-IP chaining requires two steps: the first IP writes its entire output to a bank, and only after completion can the second IP read the data back. This incurs redundant memory traffic and adds latency while the consumer waits for the producer to finish.

To avoid this, we introduce Forwarding Units (FWUs) in the slave tree. An FWU implements a direct, buffered connection between its write and read interfaces, allowing two master units to exchange data without touching the banks. Since the FWU is also memory-mapped like any other slave, the producer CU issues a write to the FWU's address range, and the consumer CU issues a read to the same range. When both sides are ready, the FWU handshakes the two requests and forwards data from its write side to its read side, preserving ordering while bypassing the banks.

The execution wrapper's major focus is to support accelerator IPs operating at higher efficiency than when connected as external peripherals to the core. Low-level optimizations target latency in IP-memory and IP-IP communication and throughput maximization. However, a key limitation of the current state is that the core, co-processor, and all accelerator IPs operate at the same clock frequency. As shown in [30], the core essentially limits the maximum frequency of the entire system. Since accelerator IPs are application-specific, it is reasonable for them to run at higher frequencies than the general-purpose RISC-V core. Moreover, the interconnect's scalability limits cannot be evaluated if the system's critical path lies outside the co-processor.

To remove this bottleneck, the design is extended so that the execution unit (interconnect network and CUs) runs in a separate clock domain. The core and the co-processor wrapper remain in one domain, while the execution unit runs on a second domain, driven by `clk_ex`. The value of `clk_ex` is selected based on the maximum achievable frequency of the accelerator IPs or the critical path of the interconnect, depending on the network size.

To realize the dual-clock design, the communication interfaces between the co-processor wrapper and the execution unit must be considered. These fall into two categories. In the first category, the register operands (`rs1`, `rs2`) are provided by the core via the *issue* interface of the XIF, and the destination register value (`rd`) is returned

to the core via the *result* interface. This category also includes the acknowledgments and handshake signals that the execution unit presents to the XIF interfaces and the internal control and configuration signals generated by the co-processor wrapper and its decoder. These signals are commonly registered locally within the co-processor, with the wrapper governing the timing of control and data delivery to the execution unit. Consequently, crossing between the two clock domains for these signals is straightforward and requires minimal intervention.

The second category consists of the memory-transfer interfaces: the XIF *memory* and *memory result* channels, and the DTCM interface of the core's data bridge. These interfaces use request/response handshakes, ultimately driven by the RISC-V core. To ensure predictable handshake semantics across the two clock domains, asynchronous FIFO buffers are placed at each clock-domain crossing. Specifically, we use the Xilinx `xpm_fifo_async` primitive, which naturally infers Block RAMs (BRAMs) with true dual-clock synchronization. Each interface (both DTCM and XIF) is split into a request channel and a response channel, and each channel is assigned a dedicated asynchronous FIFO. On the request channel, the enable, address, and write data signals are packed and pushed into the FIFO in the core domain (`clk`), and popped in the co-processor execution domain (`clk_ex`). On the response channel, the valid flags and read data are packed and pushed in the execution domain (`clk_ex`), and popped in the core domain (`clk`). However, when the packed response is popped on the core side, the valid signals are not held high, but are instead pulsed with respect to `clk_ex`. This is done to avoid multi-cycle valid signals of the response with respect to the core `clk`, and the handshake is performed only when data is valid and both clocks are high.

4 Programming Methodology

This section describes the integration of the custom instructions into the RISC-V compiler toolchain and the programming methodology used to control the co-processor and the accelerator IPs. Also, this section presents the C-level macros that abstract the implementation details of the custom instructions.

4.1 Custom Instructions Integration

The RISC-V GNU toolchain [28] is utilized in this work to integrate the custom instruction set and compile the RISC-V C program code. Rather than modifying the assembler, compiler macros are employed to define custom instructions as inline assembly code. These macros are shown in Listing 1.

Additionally, the library files of the RISC-V toolchain are modified to include the above instructions as R-type encodings. Custom masks are defined per instruction to program predefined fields such as opcode and funct3, which the co-processor uses to identify the instruction. The masks also disable specific fields when unused (e.g., funct7 in load/store instructions, or rs1 in the augmented CU instructions). With these definitions in place, the instructions can be invoked directly from C via the macro functions shown earlier.

With this extension's augmented LSU, the co-processor's scratchpad banks are mapped into the core's address space. The RISC-V linker script to allocate sections within the SPM address ranges is modified to enable this. The relevant additions to the linker script are shown in Listing 2.

First, the scratchpad memory partitions are declared in the linker script. The hardware base address mapped to the DTCM ports of the data bridge is `0x4000_0000`, which is used as the start address of the first SPM bank. Because the RISC-V toolchain assumes byte-addressable memory, each bank's length is specified in bytes: for a depth of 1024 words (32 bits per word), each bank spans 4 KB. In the 8×8 configuration used here, eight SPM banks are therefore defined as eight memory regions. Next, the initialization of the scratchpad sections is specified in the `SECTIONS` block. `ALIGN(4)` is used to ensure word alignment so that data is placed in every fourth byte address of `.spm_data`. Correspondingly, on the hardware side, addresses received from the DTCM interface are right-shifted by two bits, to match the software's byte addresses with the scratchpad's word addressing.

Listing 1. Inline assembly macros defining the custom ISA extension of this work.

```

1 #define xloadc(addr,len) ({ unsigned int ret; \
2   asm volatile ("xloadc %[rd], %[rs1], %[rs2];" \
3     : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len)); ret; \
4 })
5 #define xload(addr,len) ({ unsigned int ret; \
6   asm volatile ("xload %[rd], %[rs1], %[rs2];" \
7     : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len)); ret; \
8 })
9 #define xstorec(addr,len) ({ unsigned int ret; \
10  asm volatile ("xstorec %[rd], %[rs1], %[rs2];" \
11    : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len)); ret; \
12 })
13 #define xstore(addr,len) ({ unsigned int ret; \
14  asm volatile ("xstore %[rd], %[rs1], %[rs2];" \
15    : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len)); ret; \
16 })
17 #define xcfc(id,addr,len) ({ unsigned int ret; \
18  asm volatile ("xcfc %[f7], %[rd], %[rs1], %[rs2];" \
19    : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len), [f7] "n" (id)); ret; \
20 })
21 #define xcmp(id,addr,len) ({ unsigned int ret; \
22  asm volatile ("xcmp %[f7], %[rd], %[rs1], %[rs2];" \
23    : [rd] "=r" (ret) : [rs1] "r" (addr), [rs2] "r" (len), [f7] "n" (id)); ret; \
24 })
25 #define xlda(id,len) ({ unsigned int ret; \
26  asm volatile ("xcfc %[f7], %[rd], %[rs2];" \
27    : [rd] "=r" (ret) : [rs2] "r" (len), [f7] "n" (id)); ret; \
28 })
29 #define xcfcga(id,len) ({ unsigned int ret; \
30  asm volatile ("xcmp %[f7], %[rd], %[rs2];" \
31    : [rd] "=r" (ret) : [rs2] "r" (len), [f7] "n" (id)); ret; \
32 })

```

With the SPM address space now visible to the C program, data can be loaded directly into the SPM at program initialization using GCC section attributes.

4.2 Co-processor Modes of Operation

Next, this subsection demonstrates how the custom instructions are used to operate CUs in the co-processor. Listing 3 and 4 show two examples of executing two IPs sequentially.

In Listing 3 (as in [30]), an array `in_arr` in the core's data memory is loaded to `src_addr` in the co-processor SPM using `XLOADC` and `XLOAD`. CU 1 is then configured and begins execution on the array stored at `src_addr`. To detect completion, the core probes CU 1 with `XCFG` until a valid status is returned. Next, CU 2 is configured to consume the output of CU 1 stored at `temp_addr`. Finally, `XSTOREC` and `XSTORE` store the output from `dst_addr` back to the core's data memory at `out_arr`. Note that the LSU is denied access to any SPM bank currently in use by a CU; therefore, probing CU 2 is not required, as the LSU naturally waits until the bank becomes available before performing the store.

Listing 2. Modifications to the linker script.

```

1 MEMORY
2 {
3     SPM_RAM0(rw): ORIGIN = 0x40000000, LENGTH = 4k
4     SPM_RAM1(rw): ORIGIN = 0x40001000, LENGTH = 4k
5     SPM_RAM2(rw): ORIGIN = 0x40002000, LENGTH = 4k
6     SPM_RAM3(rw): ORIGIN = 0x40003000, LENGTH = 4k
7     SPM_RAM4(rw): ORIGIN = 0x40004000, LENGTH = 4k
8     SPM_RAM5(rw): ORIGIN = 0x40005000, LENGTH = 4k
9     SPM_RAM6(rw): ORIGIN = 0x40006000, LENGTH = 4k
10    SPM_RAM7(rw): ORIGIN = 0x40007000, LENGTH = 4k
11 }
12
13 SECTIONS
14 {
15     __SPM_RAM0_START = ORIGIN(SPM_RAM0);
16     __SPM_RAM0_SIZE = LENGTH(SPM_RAM0);
17     //Only mapping for SPM_RAM0 is shown here
18     .spm_ram0_init_start : ALIGN(4)
19     {
20         __DTCM0_INIT_START = .;
21     } >ROM
22
23     .spm0_ram : ALIGN(4)
24     {
25         __DTCM0_START = .;
26         *(.spm_data_0);
27     } >SPM_RAM0 AT>ROM
28
29     .spm_ram0_init_end :
30     {
31         __DTCM0_INIT_END = .;
32     } > ROM
33 }

```

The major drawback of the previous method is the extra latency and memory traffic incurred by issuing load/store instructions during program execution. Listing 4 shows an optimized version using the augmented LSU, which stores `in_arr` directly to the SPM at program initialization. Here, the GCC attribute `__attribute__((section("...")))` is used to place `in_arr` and `out_arr` in SPM sections, avoiding runtime XLOAD/XSTORE entirely. For an input length of 1024, this approach achieves a 3.35× speedup during execution. Moreover, because the core can access the SPM address space in parallel with co-processor execution, any latency from initializing or reading the SPM can be masked with further operations on the co-processor.

The third example demonstrates the benefit of using the FWU to chain two IPs. In the previous examples, the core had to probe CU 1 with XCFG because CU 2 could not consume results until CU 1 finished writing them to memory. As shown earlier, the FWU enables direct forwarding from one CU to the next. Listing 5 illustrates this setup as presented in [30], where CU 2 consumes CU 1’s output stream without bouncing through memory.

The key change is that CU 1’s output and CU 2’s input both target the FWU mapped at `FWU_ADDR`, which is assigned to the address range of slave unit 2. Both CUs are configured first, and execution proceeds in parallel. As soon as CU 1 produces data, the FWU forwards each word directly to CU 2 without touching the SPM, enabling true operation chaining. For an input length of 1024, this achieves a further 1.49× speedup.

Listing 3. Example code using the RISC-V custom instructions to control the co-processor [30].

```

1 int main_0(void){
2     int in_arr[1024], out_arr[1024];
3     unsigned int src_addr, temp_addr, dst_addr;
4     //load in_arr to src_addr
5     status = xloadc(&in_arr, 1024);
6     status = xload(src_addr, 1024);
7     //configure CU 1 and begin execution
8     status = xcfig(1, src_addr, 1024);
9     status = xcmp(1, temp_addr, 1024);
10    //probe CU 1
11    do { status = xcfig(1, src_addr, 1024); } while (status != 1);
12    //configure CU 2 and begin execution
13    status = xcfig(2, temp_addr, 1024);
14    status = xcmp(2, dst_addr, 1024);
15    //store output of CU 2 to out_arr
16    status = xstorec(&out_arr, 1024);
17    status = xstore(dst_addr, 1024);
18    return 0;
19 }

```

Listing 4. Example code showing the benefits of the augmented LSU.

```

1 int main_0(void){
2     int in_arr[1024] __attribute__((section(".spm_data_0")));
3     int out_arr[1024] __attribute__((section(".spm_data_1")));
4     unsigned int temp_addr;
5     //in_arr already in SPM
6     //configure CU 1 and begin execution
7     status = xcfig(1, &in_arr, 1024);
8     status = xcmp(1, temp_addr, 1024);
9     //probe \ac{cu} 1
10    do { status = xcfig(1, src_addr, 1024); } while (status != 1);
11    //configure CU 2 and begin execution
12    status = xcfig(2, temp_addr, 1024);
13    status = xcmp(2, &out_arr, 1024);
14    //out_arr already in SPM
15    return 0;
16 }

```

4.3 Unified Software Macros

To abstract the implementation details of the custom instructions and simplify co-processor programming, we define C macros that group the most common operations into concise functions. The first set of macros (as in [30]) is shown in Listing 6.

These macros encapsulate the primary LSU and CU instructions into single-call functions, so the user provides only the required parameters rather than using each custom instruction individually.

A second set of macros targets the case where the application streams data directly between the core's data memory and a specific CU (source or sink), bypassing the SPM. This set, also provided in [30], is shown in Listing 7.

Listing 5. Example code showing the benefits of the FWU.

```

1 #define FWU_ADDR 2048
2 int main_0(void){
3     int in_arr[1024] __attribute__((section(".spm_data_0")));
4     int out_arr[1024] __attribute__((section(".spm_data_1")));
5     //configure CU 1 and CU 2
6     status = xcfig(1, &in_arr, 1024);
7     status = xcfig(2, FWU_ADDR, 1024);
8     //Begin execution for both CUs
9     status = xcmp(1, FWU_ADDR, 1024);
10    status = xcmp(2, &out_arr, 1024);
11    return 0;}

```

Listing 6. Basic software macros.

```

1 int LOAD_MEM_BANK(unsigned int src_addr_mem, unsigned int dest_addr_bank, unsigned int len) {
2     int status;
3     status = xloadc(src_addr_mem, len);
4     status = xload(dest_addr_bank, len);
5     return status;
6 }
7 int STORE_BANK_MEM(unsigned int src_addr_bank, unsigned int dest_addr_mem, unsigned int len) {
8     int status;
9     status = xstorec(src_addr_bank, len);
10    status = xstore(dest_addr_mem, len);
11    return status;
12 }
13 int EX_CU_{cu_id}( unsigned int src_addr, unsigned int dest_addr,
14    unsigned int len_in, unsigned int len_out) {
15    int status;
16    status = xcfig(cu_id, src_addr, len_in);
17    status = xcmp(cu_id, dest_addr, len_out);
18    return status;
19 }

```

These macros leverage the FWU to forward data directly between a target CU and the LSU, avoiding temporary storage in the SPM. In this flow, the CUs are first configured to stream to/from the FWU. The LSU is configured to load from or store to the same FWU address range. When the load/store begins and data becomes available, the FWU forwards each word between the CU and the LSU, eliminating the memory bounce.

The final macro presented here programs the augmented CU for memory realignment using the address queue, shown in Listing 8.

The parameters for this macro mirror those of EX_CU, except that the source argument now points to an array of memory indices describing the input-stream access pattern for the IP. This flow requires four instructions, unlike EX_CU. As described in the implementation section, XCFIG and XLDA first load the address pattern into the CU's address queue, then XCFIGA and XCMP configure the CU to use the queue for reads and begin streaming to the IP. From the user's perspective, the only additional step is to generate the address pattern, and the execution still reduces to a single macro call.

Working towards automating the integration of the custom IPs in both hardware and software, Python scripts are used to generate both the hardware architecture and software macros.

Listing 7. Direct stream macros.

```

1 #define FWU_ADDR 2048
2 int EX_CU_{cu_id}_DIRECT_INPUT(unsigned int src_addr_mem, unsigned int dest_addr_bank,
3 unsigned int len_in, unsigned int len_out) {
4     int status;
5
6     status = xcfg(cu_id, FWU_ADDR, len_in);
7     status = xcmp(cu_id, dest_addr_bank, len_out);
8     status = xloadc(src_addr_mem, len_in);
9     status = xload(FWU_ADDR, len_in);
10    return status;
11 }
12 int EX_CU_{cu_id}_DIRECT_OUTPUT(unsigned int src_addr_bank, unsigned int dest_addr_mem,
13 unsigned int len_in, unsigned int len_out) {
14     int status;
15
16     status = xcfg(cu_id, src_addr_bank, len_in);
17     status = xcmp(cu_id, FWU_ADDR, len_out);
18     status = xstorec(FWU_ADDR, len_out);
19     status = xstore(dest_addr_mem, len_out);
20    return status;
21 }

```

Listing 8. Aligned stream macros.

```

1 int EX_CU_{cu_id}_aligned(unsigned int *src_addr, unsigned int dest_addr,
2 unsigned int len_in, unsigned int len_out) {
3     int status;
4     status = xcfg(cu_id, &src_addr, len_in);
5     status = xlda(cu_id, len_in);
6     status = xcfga(cu_id, len_in);
7     status = xcmp(cu_id, dest_addr, len_out);
8     return status;
9 }

```

- When the list of IPs to integrate into the co-processor is given, the scripts can automatically interface them with individual CUs in the interconnect network.
- The scripts also accept parameters such as the network size (number of master and slave units) and the number and locations of FWUs, to interface with the Vivado TCL console and generate the co-processor architecture, thereby producing the complete SoC for implementation.
- On the software side, the scripts generate an application template using the modified RISC-V GNU toolchain, containing the previously described software macros. Individual functions are generated for each existing CU, by modifying the `cu_id` parameter of the macros as shown in Listing 6 and 8.

Using these scripts requires minimal effort to integrate the accelerators into the co-processor system and pushes towards a seamless plug-and-play integration.

Table 5. Resource utilization of design *without IPs* on RFSoc 4x2.

Component	LUTs	CLB Registers	Block RAM	DSPs
Total available	425280	850560	1080	4272
RISC-V Core	4,383	2,346	0	3
Instr. & Data mem.	543	536	78.5	0
Co-processor	8,117	4,832	15.5	0
IP Cluster	100	114	7	0
Slave Tree (SPM)	128	14	7	0
LSU Controller	307	173	0	0
CU Controller	339	530	0	0
Router Tree	2,016	0	0	0
Arbitration Tree	2,128	168	0	0
Complete SoC	14,755	8,560	95	3

The Complete SoC includes the Core, memories, Co-processor and other components.

5 Experimental Evaluation

In this section, the proposed implemented design is evaluated on an FPGA. First, a representative configuration of the entire SoC is prototyped on the FPGA to report resource utilization and power consumption. Then, a performance analysis is conducted on the co-processor architecture to report throughput, latency, and system scalability metrics. Finally, the DNN ShuffleNet-V2 is implemented as a use case for our system.

5.1 Prototyping and Resource Utilization

The hardware architecture, designed in Verilog HDL, is implemented on an AMD/Xilinx RFSoc 4x2 evaluation board using Vivado 2024.2. The entire SoC, including the core and the co-processor, is implemented on the FPGA fabric and connected to instruction and data memories via a 32-bit AXI-4 bus. The instruction and data memories are configured as Xilinx BRAM blocks of 4 KB and 1024 KB, respectively. The baseline co-processor configuration is an 8x8 network, where the slave tree comprises seven scratchpad banks and one FWU, while the master tree comprises seven CUs and one LSU. Each scratchpad bank is a BRAM with a depth of 1024 words (32 bits each). Each memory bank is configured as a Xilinx BRAM with a depth of 1024 words, and each CU is connected as a Xilinx AXI-S FIFO IP, also with a depth of 1024 words. Each CU is attached to a Xilinx AXI4-Stream FIFO IP with depth 1024 words, serving as a stand-in for stream-based accelerators, whose resource cost is application-dependent and not the focus here. Implementation-time optimizations that aggressively reshape hierarchy are minimized to better reflect the intrinsic resource cost of the co-processor architecture. The resulting utilization and timing are reported in Table 5, and the improvements over our prior design[30] are summarized in the rest of this subsection.

First, the modifications to the LSU and CU are reflected in the table. Because the LSU can now bypass the XIF for core-co-processor transfers, its design has been simplified, and a reduced LSU is maintained in this work. Adding the address-queue logic is also visible, as the per-CU utilization nearly doubled compared to our previous work[30].

Second, the routing and arbitration trees significantly reduce resources due to dedicated switch-level optimizations introduced in this extension. These changes lower the area and shorten the interconnect critical path, enabling higher co-processor frequencies.

A corresponding power reduction is observed at 100 MHz, which is the operating point limited by the core, used in our previous work[30]. Under the same 8×8 configuration with FIFO-backed CUs, total dynamic power for the co-processor decreases from 67 mW (prior work) to 49 mW (this work), an 18 mW (approx. 27%) reduction.

5.2 Performance Evaluation

The first evaluation measures the throughput gain from chaining multiple IPs using FWUs. As explained earlier, without an FWU, forwarding data from one IP to another requires the consumer to wait until the producer has finished writing its output to a memory bank. In software, the core probes the producer CU with XCFG until a valid status is returned before starting the consumer. With an FWU, both CUs can be launched simultaneously, and the consumer begins receiving its input stream as soon as the producer emits its first word.

We define *throughput* as the effective number of words processed per cycle, measured from the cycle when the first IP in the chain accepts its first input word to the cycle when the last IP produces its final output word. For a single IP with a non-staggered input/output pattern, the effective throughput approaches one word/cycle. For multiple IPs chained via FWUs, the effective throughput increases because several stages can process data concurrently, although the exact gain depends on the input length and stage latencies.

To visualize this effect, we configure an 8×8 network in which all CUs are FIFO IPs and instantiate the number of FWUs required by the chain length. We then measure the effective throughput under the above definition and report the results in Figure 10.

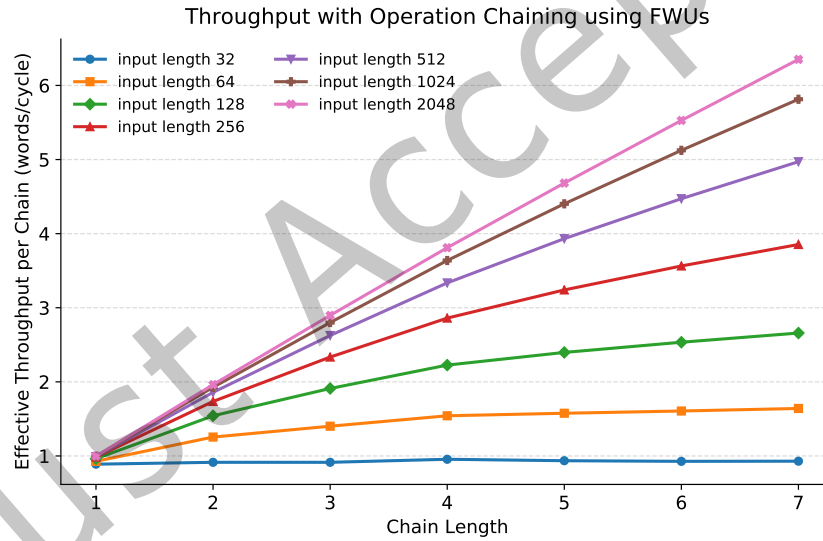


Fig. 10. Throughput of operations chaining multiple CUs together using the FWUs.

The throughput is measured across increasing CU chain lengths and multiple input sizes. Two general trends emerge: (i) throughput increases with larger input lengths, and (ii) throughput increases with longer chains. The first trend arises because the effect of the fixed overhead of issuing custom instructions (configuration, execution, and optional probing) diminishes over a larger transfer. For a chain length of 1, short inputs yield effective throughput well below one word/cycle and improve only marginally as the length grows. This also reflects the baseline throughput one would observe without FWUs. Regarding the second trend, FWUs enables multiple IPs in the chain to process data simultaneously, so longer chains achieve higher effective throughput, especially for

large inputs. Conversely, with very short inputs (e.g., length 32), instruction latency dominates, and the measured throughput barely approaches 1 word/cycle even as the chain length increases.

The second evaluation concerns the latency of executing the custom instructions. As reported in [30], the configuration and control instructions XLOADC, XSTOREC, XCFG, XCMP, XLDA, and XCFGA each have a fixed latency of 5 cycles. For the data-movement instructions XLOAD and XSTORE, overhead is measured per transferred word between the core's data memory and the co-processor's SPM. In both cases, each word incurs five cycles—XLOAD accounts for one cycle on the core side and four on the co-processor side, while XSTORE accounts for two cycles on the core side and three on the co-processor side.

In this work, we also evaluate latency from the perspective of interconnect congestion when multiple master units contend for the same slave. Here, *latency* is defined as the average number of clock cycles a master waits per word transfer to a given slave while competing with other masters. We measure this in a hotspot scenario by fixing one scratchpad bank (or FWU) as the destination and increasing the number of competing masters. The average wait cycles per word are reported in Figure 11.

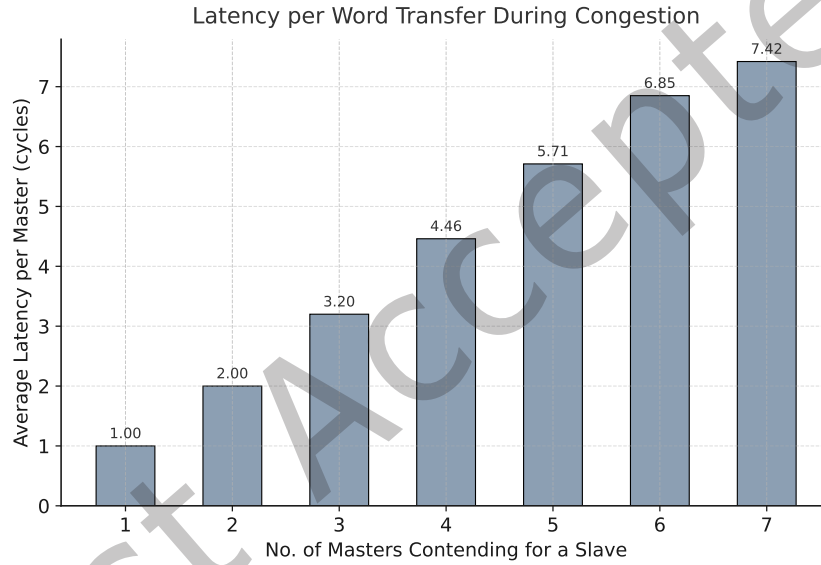


Fig. 11. Average latency per word transfer to a master unit under contention for slave access.

We evaluate congestion-induced latency on an 8×8 network with FIFOs as CUs. For each point in Figure 11, multiple IPs located at different positions in the master tree are programmed to read simultaneously from the *same* memory bank.

As a concrete example, when three IPs contend for one slave, there are $\binom{7}{3} = 21$ ways to place them in the master tree (seven available CU slots). For each placement, we measure the average latency for a fixed reference master, and then report the mean across all placements. Averaging is necessary because a master's wait time depends on where the other contenders sit in the tree.

Contention only arises at the arbitration layers. If a first-level arbiter sees two valid inputs, each contending master experiences a 2-cycle wait. If contention occurs at the second level for the same destination, the initial two masters see four cycles, while a third master that only collides at the second level still sees two cycles. Thus, the per-master latency can be two or four cycles with three contenders, depending on their relative positions.

Table 6. Timing and resource utilization for various network configurations on RFSoc 4x2.

Config.	# Masters	# Slaves	Min. Clk Period (ns)	Max Freq (MHz)	LUTs	CLB FFs	% Util.
2x2	2	2	1.6	625	1,963	1,375	0.4
2x4	2	4	1.6	625	2,151	1,391	0.5
4x4	4	4	1.8	555	3,417	2,451	0.8
4x8	4	8	2.1	476	4,521	2,513	1.0
8x8	8	8	3.0	333	8,101	4,729	1.9
8x16	8	16	3.4	294	12,677	4,913	3.0
16x16	16	16	4.7	212	24,573	9,537	5.8
16x32	16	32	5.2	192	43,197	10,289	10.1
32x32	32	32	6.1	164	85,993	20,305	20.2

This behavior holds across all cases where multiple masters contend for the same slave. As shown in Figure 11, the average latency per word grows rapidly as more masters target the hotspot, peaking when six of the seven CU slots contend for a single slave. Interestingly, the measured average then drops at the next point, and this could be because the eighth slot in the master tree is reserved for the LSU, which remains idle during program execution and therefore does not contend with its adjacent CU.

The third and final evaluation analyzes the scalability of the interconnect network. The objective is to determine the maximum operating frequency attainable for different network sizes and to identify the execution unit's practical limits. This is particularly relevant because the interconnect is implemented in combinational logic. Therefore, the end-to-end path delay between IPs and the scratchpad memory must remain below the clock period.

This study also indicates how resource usage grows as the network scales, guiding the largest feasible configuration of the execution unit. Such an analysis is possible because of the system's dual clock domain, which allows the execution unit, which hosts the interconnect network, the IPs, and scratchpad memory, to run at a frequency independent of the core. With asynchronous FIFOs at the execution-unit boundary, all combinational paths across the clock-domain crossing are removed, achieving complete timing isolation for the execution unit and allowing its frequency to be swept without affecting the core.

For this evaluation, the execution unit is configured with all CUs attached to FIFO IPs (depth 1024 words) and all slave banks sized to 1024 words. At least one FWU is included in every configuration. Network sizes are swept from 2x2 up to 32x32, restricting to cases where the number of slave units is *greater than or equal to* the number of CUs so that every CU can operate in parallel (i.e., configurations with fewer slaves than CUs are skipped). For each configuration, we determine the minimum achievable clock period and record the resource utilization, and the results are reported in Table 6.

The results in Table 6 show an apparent decline in maximum frequency as the network scales. The critical path does not lie in the interconnect for the 2x2 and 2x4 configurations. However, the critical path runs from a CU's FIFO output to the slave-tree input (memory bank or FWU) for all larger sizes. From the table, the 8x8 network is the largest configuration that achieves a frequency over 300 MHz. At this point, the execution unit alone consumes roughly 2% of the available CLB LUTs on the RFSoc 4x2. For larger sizes, not only does the maximum frequency drop significantly, but resource usage grows sharply, nearly doubling each step, with the 32x32 network exceeding 20% of LUT resources. Therefore, we consider 4x4 through 8x16 to be the most practical

operating points since they are lightweight in FPGA resource utilization while still offering ample CU slots and supporting high frequencies for the accelerator IPs.

5.3 Use case: ShuffleNet V2

To showcase the different features of our co-processor architecture and demonstrate its advantages in an end-to-end workload, ShuffleNet V2 [16] is used as a case study for hardware acceleration. We chose this DNN because it is smaller and better suited to edge applications, and because the model itself offers strong potential for hardware reuse and parallelism.

Since the goal of this work is not to develop the accelerators themselves, but rather to integrate and use them, HLS-based applications are targeted. Here, we adopt a toolchain based on the HiFlipVX library [11] to generate a hardware accelerator for ShuffleNet using Vitis HLS 2024.1. The toolchain generates optimized hardware implementations of neural networks from ONNX [21] models, enabling the creation of Xilinx-optimized AXI-S IPs for each ONNX node or a group of nodes.

The ONNX model for ShuffleNet-V2 is obtained from the ONNX model zoo [22]. At a first glance at the ONNX graph [22], we observe that, at a global level, multiple sequences in the model are repeated. Therefore, the graph is split into seven unique segments. When the seven segments are chained together as shown in Figure 12 (a), the full ShuffleNet is formed. The most interesting segments for us are segments 2, 4, and 6, because they contain multiple instances of identical blocks. For example, Figure 12 (b) shows a block within segment 4 that repeats seven times. Therefore, when implementing this segment in our co-processor, only one instance of the repetitive block needs to be synthesized, and execution of the full segment can be handled through our custom instructions in software.

The ONNX graphs are fed to the toolchain as input, and the toolchain synthesizes the entire graph as a single top-level AXI-S IP, with an 8-bit input and output stream at each node. The IPs are synthesized for the AMD/Xilinx RFSoc 4x2 board, targeting a frequency of 100 MHz. The toolchain first optimizes the ONNX graph, then generates each graph node as an AXI-S IP, and finally chains them together using stream FIFOs as graph edges. Various graph optimization methods are applied to improve performance and meet the streaming requirements of the hardware design. Each node consists of a computation engine (such as convolution, pooling, or activation) generated using the HLS-based HiFlipVX library [11], coupled with small local buffers to store reusable weights and biases, or larger BRAMs to hold all the weights and biases required by the node. The toolchain also automatically tunes the graph for performance by vectorizing the larger and slower kernel nodes.

The generated IPs for each ShuffleNet segment serve as a baseline for comparison. However, synthesizing an entire segment as a single IP provides no control over the individual kernels within the segment and also leads to underutilization of hardware. In particular, because there is a single unified input port, BRAMs store the weights and biases of all kernels within the IP, and internal FIFO buffers chain the kernel IP cores. We therefore show how this NN can be optimized using our system with minimal effort, leveraging its various features. The IP cores of the NN kernels are not further optimized, as the functionality of the IPs is not the focus of this work. Rather, we focus on providing more control over each kernel and improving their integration.

To implement segments such as those presented in Figure 12 in our system, we leverage the inherent parallelism and chaining capabilities, synthesizing individual kernel functions (e.g., convolution, ReLU) as AXI-S IPs. The FWUs are used to chain the IPs, thereby removing the inter-IP FIFOs inserted by the toolchain.

Since our co-processor already includes scratchpad memory banks, the internal weight memories of the IPs can be removed, and the weights can instead be stored in the scratchpad memory. Furthermore, since our co-processor supports IPs with multiple input/output streams, the HLS toolchain is configured to add an additional stream port for loading weights and biases from the co-processor's scratchpad memory. The scatter unit and datawidth converter presented in [11] are used to collect weights and biases from a unified input stream port and distribute

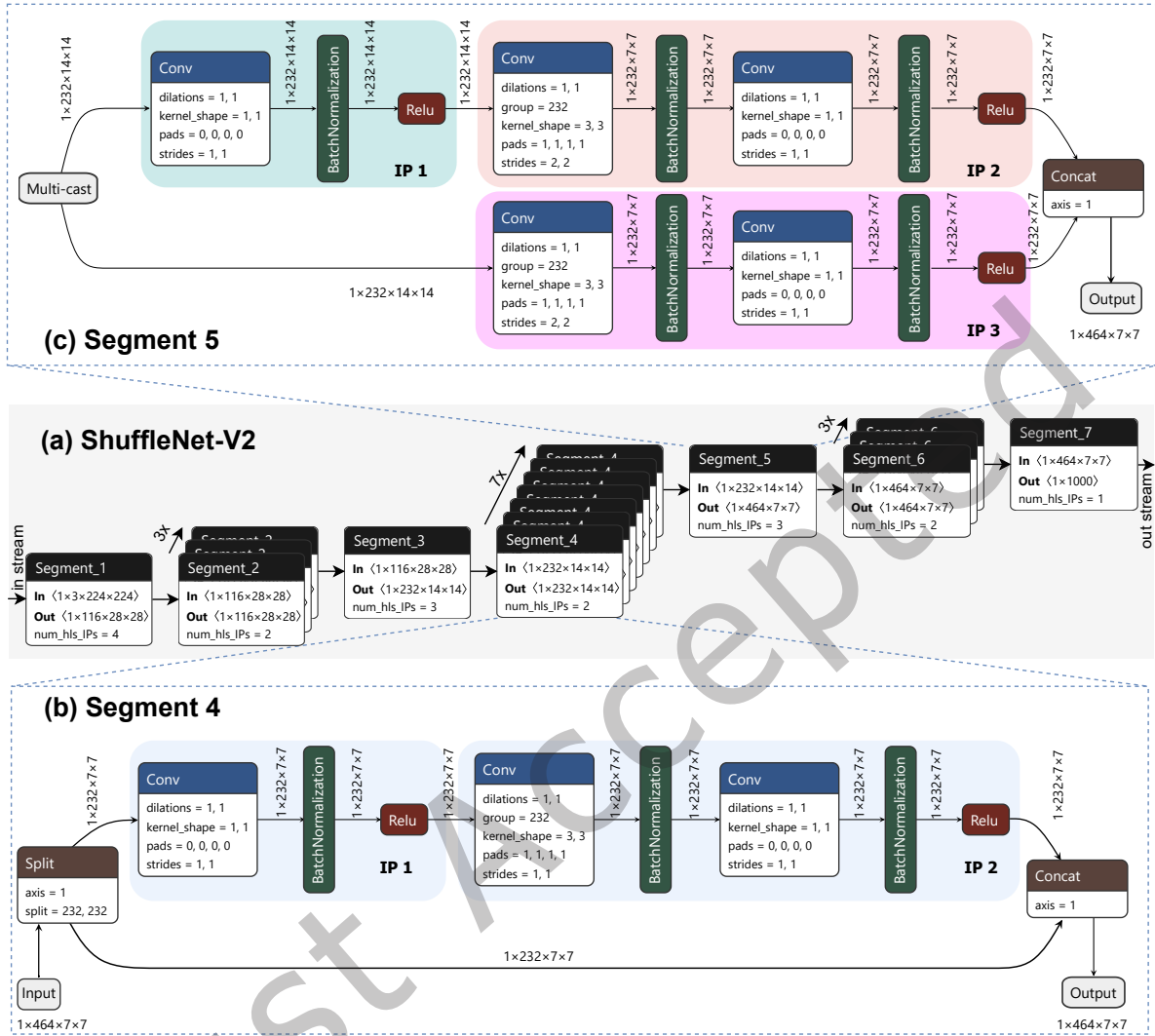


Fig. 12. Manually partitioned segments of the ShuffleNet-V2 ONNX graph [22]: (a) Sequence of segments connected to form the full NN, highlighting input/output sizes and number of HW IPs in each segment. (b) Repetitive block of Segment 4 and (c) Segment 5, highlighting the ONNX nodes synthesized as IPs using HLS.

them to the individual weight and bias buffers. Thus, each kernel IP is generated with two input streams—one for input data and the other for weights and biases—and one output stream. By assigning dedicated custom instructions to each stream port, we provide fine-grained control over data streaming and weight loading.

In special cases, such as in Figure 12 (b) and (c), where the stream splits into two parallel streams and is concatenated later, dedicated splitter and multicast units developed in [20] are used by the toolchain. When these parallel streams have different latencies, there is a risk of a deadlock; therefore, the toolchain adds delay FIFOs, implemented using multiple BRAMs or a URAM, to the shorter chain. However, in our system, since the inputs

and outputs can be stored in the SPM, inputs are available on demand, and the split and multicast operations can be performed in software by configuring the custom instructions of each stream.

Using the methods described above, we implement the kernels in our co-processor and execute the ShuffleNet-V2 segments. The co-processor is configured as an 8×8 network. Each SPM bank is configured with a 32-bit word width and a 16k-word depth to accommodate large input and weight matrices. The number of FWUs varies depending on the segment. Also, data-width converters are added to the data stream ports to make the 8-bit IPs compatible with our 32-bit network.

To demonstrate how the different segments of the network are implemented in our co-processor, Figures 12 (b) and (c) highlight the groups of ONNX nodes that are synthesized as standalone IPs. Figure 13 shows, as an example, a high-level visualization of how the implementation of segment 5 is realized in our system. All the IPs are connected to the interconnect network as master units with two CU interfaces, a data interface for input and output data, and a weights interface for layer weights and biases. To form the dataflow of the segment as shown in Figure 12 (c), IPs within the same chain are connected through FWUs, and the inputs to the two parallel chains are fed from the same SPM bank, as shown in Figure 13. This is achieved in software by using custom instructions to configure the input and output destinations of the IPs' data interfaces, programmable at run time. The weight interfaces are also similarly configured to fetch the corresponding weights and biases of each IP from other memory banks. The remaining operations, such as split and concatenation, are also implemented in software by managing input and output addresses to the IPs, whereas the multicast operation is inherently handled by the arbitration network in hardware. This methodology is used to implement all segments of the ShuffleNet, synthesizing HLS IPs, connecting them in hardware, and using custom instructions to program their dataflow in software.

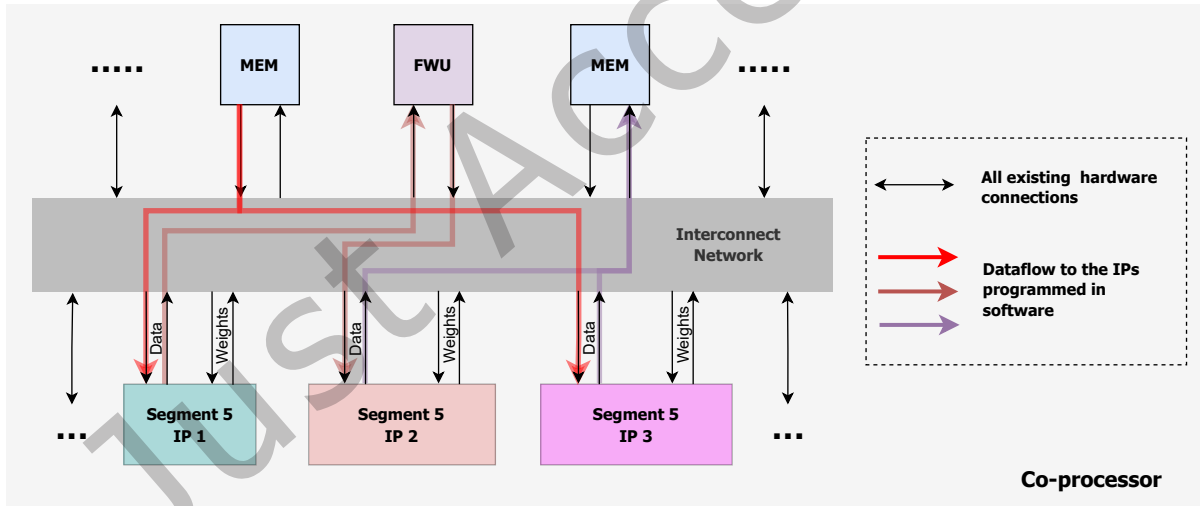


Fig. 13. Example of realizing Segment 5 in the co-processor, highlighting the hardware connections of the IPs through two CU interfaces (data and weights), and the dataflow within the segment, programmed through custom instructions.

The results of implementing ShuffleNet-V2 in our co-processor are presented in Figure 14. The response latency of each segment when implemented as standalone IPs, shown in Figure 14 (a), compared to when implemented as kernels in our co-processor. Since network latency strongly depends on the internal datapath and input pattern, we define response latency as the time between the first input to a segment and the last output from the segment.

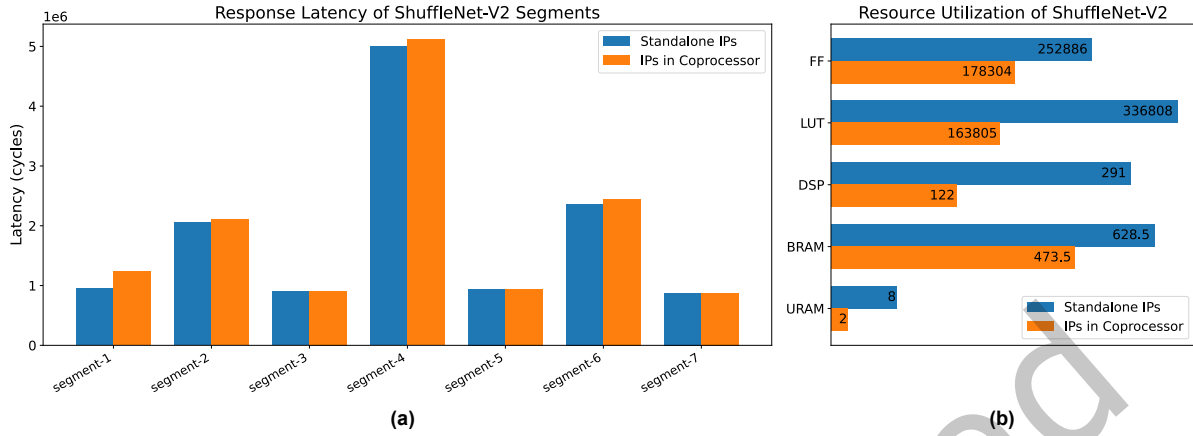


Fig. 14. Implementation of ShuffleNet-V2 in the co-processor and as standalone IPs: (a) Response latency of each segment, (b) Resource utilization.

Since the IPs' datapaths are not modified, we expect the latency of each segment in our co-processor to be comparable to, or slightly higher than, that of the standalone IPs. Figure 14 (a) reflects this expectation, but it is promising that our co-processor does not add a significant amount to the latency. However, our work primarily demonstrates improvements in hardware utilization for ShuffleNet IPs when implemented in the co-processor. As shown in Figure 14 (b), when the DNN is implemented using the features of our co-processor described above, there is a significant reduction in resource utilization. The 29% reduction in FFs and 51% reduction in LUTs can be attributed to two factors: first, the reuse of repetitive blocks, and second, the removal of hardware data-management units such as split and multicast. The corresponding removal of buffers associated with the split and multicast units reduces URAM usage. The nearly 25% reduction in BRAMs is attributed to streaming weights from the SPM rather than storing all weights inside the IP. The 58% reduction in DSP usage, however, is more dependent on Vitis HLS, which determines whether to use LUTs and FFs or DSPs to implement the computation logic depending on the spatial data patterns.

Overall, these results show that our co-processor enables significant resource savings for ShuffleNet-V2 with minimal latency impact, while maintaining fine-grained control over kernel IPs via custom instructions without modifying their datapaths.

6 Conclusion

This work presents a modular RISC-V co-processor capable of hosting a scalable number of AXI4-Stream accelerator IPs and interfacing them with a multi-bank scratchpad memory through a single-cycle logarithmic interconnect. We also support the hardware architecture with a compact custom instruction set, which integrates with the RISC-V GNU toolchain through inline assembly macros, and also supports data placement in the scratchpad memory. Further, C-level macros are provided to abstract the implementation details of the custom instructions and enable configuration and execution of the co-processor hardware through simple functions.

In the hardware datapath, FWUs are introduced to enable CU-to-CU streaming without bouncing through memory, and the CU is augmented with an address queue and new instructions to enable memory realignment via preloaded index sequences. To decouple the co-processor from the rest of the system, the execution unit is separated into its own clock domain and asynchronous FIFOs are employed for the clock-domain crossings.

Our design prototype on an AMD/Xilinx RFSoc 4x2 FPGA shows a reduction in resource utilization compared to the previous work presented in [30], and under the same 8x8 configuration used previously, co-processor dynamic power drops by 18 mW, showing approximately a 27% reduction. Improvements in our hardware architecture reflect in software performance, showing that: (i) chaining CUs via FWUs increases effective throughput by overlapping stages, and (ii) bypassing the co-processor interface to provide the core direct access to the SPM shows a significant speedup during runtime. Performance analysis included measuring the throughput gains when using the FWUs, and per-master latency of word transfers under congestion in the interconnect network. A scalability study across different network sizes identified that 4x4 to 8x16 networks are the most practical configurations that balance frequency, resource utilization, and available CU slots for IP integration. It also showed that 8x8 remains this platform's largest configuration, exceeding 300 MHz.

Finally, we demonstrated the effectiveness of our system using ShuffleNet-V2 as a realistic use case for integrating stream-based HLS IPs. This evaluation showed that leveraging the features of our co-processor achieves a reduction in hardware resource utilization when compared to standalone IP implementations, by 29% for FFs, 51% for LUTs and 25% for BRAMs, while maintaining comparable latency and enabling fine-grained control over the individual kernel IPs.

Future scope for this work includes enabling dynamic partial reconfiguration for our architecture targeting FPGAs, allowing the accelerator IPs connected to the CUs to be seamlessly swapped out or reconfigured at runtime, rather than during design time.

Acknowledgements

The research work presented in this paper was partially funded by the German Research Foundation ("Deutsche Forschungsgemeinschaft") (DFG) under Project-ID 287022738 TRR 196 for Project S05. The authors would like to thank the Federal Ministry of Research, Technology, and Space (BMFTR) for its support as part of the research program Communication Systems "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KIS2413K. Furthermore, this work was partially funded by the German Federal Ministry of Research, Technology, and Space (BMFTR) as part of the DAKORE (Datenfunknetz mit Adaptivhardware und KI-Optimierung zur Reduktion des Energieverbrauchs) project under grant agreement number 16ME0433K. As well as by the German Federal Ministry of Research, Technology, and Space (BMFTR) by funding the competence center for Big Data and AI "ScaDS.AI Dresden/Leipzig".

References

- [1] Ryan Albert Antonio, Joren Dumoulin, Xiaoling Yi, Josse Van Delm, Yunhao Deng, Guilherme Paim, and Marian Verhelst. 2025. An open-source hw-sw co-development framework enabling efficient multi-accelerator systems. In *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–7. doi:10.1109/ISLPED65674.2025.11261784
- [2] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. 2016. The rocket chip generator. EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.pdf>.
- [3] Gianluca Bellocchi, Alessandro Capotondi, Francesco Conti, and Andrea Marongiu. 2021. A RISC-V-based FPGA Overlay to Simplify Embedded Accelerator Deployment. In *2021 24th Euromicro Conference on Digital System Design (DSD)*. IEEE, 9–17. doi:10.1109/DSD53832.2021.00011
- [4] Danilo Cammarata, Matteo Perotti, Marco Bertuletti, Angelo Garofalo, Pasquale Davide Schiavone, David Atienza, and Luca Benini. 2025. Quadrilatero: A RISC-V programmable matrix coprocessor for low-power edge applications. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers: Workshops and Special Sessions*. 66–69. doi:10.1145/3706594.3726978
- [5] Michele Caon, Clément Choné, Pasquale Davide Schiavone, Alexandre Levisse, Guido Masera, Maurizio Martina, and David Atienza. 2025. Scalable and risc-v programmable near-memory computing architectures for edge nodes. *IEEE Transactions on Emerging Topics in Computing* (2025). doi:10.1109/TETC.2025.3555869
- [6] Tao Chen, Shreesha Srinath, Christopher Batten, and G Edward Suh. 2018. An architectural framework for accelerating dynamic parallel algorithms on reconfigurable hardware. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE,

- 55–67. doi:10.1109/MICRO.2018.00014
- [7] Emilio G. Cota, Paolo Mantovani, Giuseppe Di Guglielmo, and Luca P. Carloni. 2015. An Analysis of Accelerator Coupling in Heterogeneous Architectures. In *Proceedings of the 52nd Annual Design Automation Conference* (San Francisco, California). Article 202, 6 pages. doi:10.1145/2744769.2744794
 - [8] Alessandra Dolmeta, Stefano Di Matteo, Emanuele Valea, Mikael Carmona, Antoine Loiseau, Maurizio Martina, and Guido Masera. 2025. TYRCA: A RISC-V Tightly-coupled accelerator for Code-based Cryptography. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7. doi:10.23919/DATE64628.2025.10993202
 - [9] Davide Giri, Kuan-Lin Chiu, Guy Eichler, Paolo Mantovani, N Chandramoorth, and Luca P Carloni. 2020. Ariane+ NVDLA: Seamless third-party IP integration with ESP. Workshop on Computer Architecture Research with RISC-V (CARRV), Retrieved from https://www.cs.columbia.edu/~luca/research/giri_carrv20.pdf.
 - [10] Carsten Heinz, Jaco Hofmann, Jens Korinth, Lukas Sommer, Lukas Weber, and Andreas Koch. 2021. The TaPaSCo open-source toolflow: For the automated composition of task-based parallel reconfigurable computing systems. *Journal of Signal Processing Systems* 93, 5 (2021), 545–563. doi:10.1007/s11265-021-01640-8
 - [11] Lester Kalms, Pedram Amini Rad, Muhammad. Ali, Arsany Iskandar, and Diana Göhringer. 2021. A Parametrizable High-Level Synthesis Library for Accelerating Neural Networks on FPGAs. *Journal of Signal Processing Systems (JSPS)* 93, 5 (05 2021), 513–529. doi:10.1007/s11265-021-01651-5
 - [12] Ahmed Kamaleldin, Habib Aouinti, and Diana Göhringer. 2026. ProCon-V: A Programmable Tightly Coupled Convolution Accelerator Based on RISC-V Custom Instructions for Edge Devices. *IEEE Trans. Comput.* 75, 5 (2026), 1991–2006. doi:10.1109/TC.2026.3663365
 - [13] Ahmed Kamaleldin and Diana Göhringer. 2022. AGILER: An Adaptive Heterogeneous Tile-Based Many-Core Architecture for RISC-V Processors. *IEEE Access* 10 (2022), 43895–43913. doi:10.1109/ACCESS.2022.3168686
 - [14] Ahmed Kamaleldin, Salma Hesham, and Diana Göhringer. 2020. Towards a Modular RISC-V Based Many-Core Architecture for FPGA Accelerators. *IEEE Access* 8 (2020), 148812–148826. doi:10.1109/ACCESS.2020.3015706
 - [15] Andreas Kurth, Björn Forsberg, and Luca Benini. 2022. Herov2: Full-stack open-source research platform for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* 33, 12 (2022), 4368–4382. doi:10.1109/TPDS.2022.3189390
 - [16] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*. 116–131. doi:10.48550/arXiv.1807.11164
 - [17] Simone Machetti, Pasquale Davide Schiavone, Giovanni Ansaloni, Miguel Peón-Quirós, and David Atienza. 2025. X-HEEP: An Open-Source, Configurable and Extendible RISC-V Platform for TinyAI Applications. In *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Vol. 1. IEEE, 1–6. doi:10.1109/ISVLSI65124.2025.11130281
 - [18] Vincenzo Maisto, Stefano Mercogliano, Manuel Maddaluno, and Alessandro Cilardo. 2025. Simply-V: A RISC-V Reconfigurable Playground Soft-SoC for Open Hardware Research and Fast Prototyping. *WiPec Journal-Works in Progress in Embedded Computing Journal* 11, 1 (2025), 4–4. doi:10.64552/wipiec.v11i1.86
 - [19] Paolo Mantovani, Davide Giri, Giuseppe Di Guglielmo, Luca Piccolboni, Joseph Zuckerman, Emilio G Cota, Michele Petracca, Christian Pilato, and Luca P Carloni. 2020. Agile SoC development with open ESP. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9. doi:10.1145/3400302.3415753
 - [20] Matthias Nickel, Lester Kalms, Julian Haase, Jieyu Zhao, and Diana Göhringer. 2025. Hardware-Optimized RNN Detection for Insertion/Deletion Channels in Wireless Communication. In *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Vol. 1. 1–6. doi:10.1109/ISVLSI65124.2025.11130325
 - [21] ONNX Project Contributors. 2026. ONNX: Open Neural Network Exchange. Retrieved from <https://github.com/onnx/onnx>.
 - [22] ONNX Project Contributors. 2026. ONNX ShuffleNet Model - Vision Classification. Retrieved from <https://github.com/onnx/models/tree/main/validated/vision/classification/shufflenet>.
 - [23] OpenHW Group. 2025. Core-V eXtension Interface (CV-X-IF) Specification. Retrieved from <https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/intro.html>.
 - [24] OpenHW Group. 2025. CV32E40X Core. Retrieved from <https://github.com/openhwgroup/cv32e40x>.
 - [25] Matteo Perotti, Samuel Riedel, Matheus Cavalcante, and Luca Benini. 2025. Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025). doi:10.1109/TCAD.2025.3528349
 - [26] Matteo Perotti, Yichao Zhang, Matheus Cavalcante, Enis Mustafa, and Luca Benini. 2024. MX: Enhancing RISC-V's Vector ISA for Ultra-Low Overhead, Energy-Efficient Matrix Multiplication. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6. doi:10.23919/DATE58400.2024.10546720
 - [27] Yunhui Qiu, Yiqing Mao, Xuchen Gao, Sichao Chen, Jiangnan Li, Wenbo Yin, and Lingli Wang. 2024. Fdra: A framework for a dynamically reconfigurable accelerator supporting multi-level parallelism. *ACM Transactions on Reconfigurable Technology and Systems* 17, 1 (2024), 1–26. doi:10.1145/3614224
 - [28] RISC-V GNU Toolchain. 2025. Retrieved from <https://github.com/riscv-collab/riscv-gnu-toolchain>.

- [29] Pasquale Davide Schiavone, Simone Machetti, Miguel Peón-Quirós, Jose Miranda, Benoît Denkinger, Thomas Christoph Müller, Ruben Rodríguez, Saverio Nasturzio, and David Atienza Alonso. 2023. X-heep: An open-source, configurable and extendible risc-v microcontroller. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 379–380. doi:10.1145/3587135.3591431
- [30] Rohan Krishna Vijayaraghavan, Ahmed Kamaleldin, Matthias Nickel, and Diana Göhringer. 2025. A RISC-V Coprocessor for Seamless Integration of Stream-Based Accelerators. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 1221–1227. doi:10.1109/IPDPSW66978.2025.00199
- [31] Florian Zaruba, Fabian Schuiki, Torsten Hoeffler, and Luca Benini. 2020. Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads. *IEEE Trans. Comput.* 70, 11 (2020), 1845–1860. doi:10.1109/TC.2020.3027900